

Question 1

An engineer drops the Unity Catalog external table `corp.raw.vendor_feed` with `DROP TABLE`. No separate storage cleanup runs afterward, and the external location still exists.

What remains after the drop?

- A. The registration is removed, and Unity Catalog deletes the data files seven days later unless the table is undropped before then
- B. The table's registration and its grants remain in Unity Catalog, while the data files at the table's storage path are deleted
- C. The data files at the table's path in cloud storage remain, while the table's registration in Unity Catalog is removed entirely
- D. The data files remain, and Unity Catalog locks the table's path so that no new table can be created on those files afterward

Correct answer: C

- C. The data files at the table's path in cloud storage remain, while the table's registration in Unity Catalog is removed entirely

Explanation

Correct answer:

The data files at the table's path in cloud storage remain, while the table's registration in Unity Catalog is removed entirely

Why this is correct:

For an external table, Unity Catalog stores the registration and metadata while the customer-managed cloud path stores the actual files. `DROP TABLE` removes the catalog object; without a separate storage deletion, the external data files remain at that path.

Why the other options are wrong:

The data files remain, and Unity Catalog locks the table's path so that no new table can be created on those files afterward: The catalog registration is removed, but Unity Catalog does not impose a path lock that permanently prevents registering those files through another table. Storage ownership stays with the external-location administrator.

The table's registration and its grants remain in Unity Catalog, while the data files at the table's storage path are deleted: The external table registration does not remain after `DROP TABLE`, and the command does not delete its external files. Both clauses reverse the respective catalog and storage outcomes.

The registration is removed, and Unity Catalog deletes the data files seven days later unless the

table is undropped before then: The delayed managed-table file cleanup behavior does not apply to external data. Dropping an external registration does not schedule Unity Catalog to delete the customer's files seven days later.

References:

1. DROP TABLE | Databricks on AWS:

<https://docs.databricks.com/aws/en/sql/language-manual/sql-ref-syntax-ddl-drop-table>

Question 2

An Auto Loader stream reads JSON with an explicitly supplied schema in which quantity is INT. Some files contain quantity values such as "ten". The rescued data column is enabled with its default name.

What happens to those records?

- A. The records are kept with quantity set to NULL, because the original mismatched value is captured in the `_rescued_data` column instead
- B. The schema evolves quantity to STRING, and the stream restarts so that later records keep their text values in the quantity column as they arrive
- C. The stream fails on the first bad value, with the explicitly supplied schema turning each type mismatch into a hard parsing error
- D. The records are dropped from the output, and Auto Loader writes the rejected values to the stream's checkpoint directory for later review

Correct answer: A

A. The records are kept with quantity set to NULL, because the original mismatched value is captured in the `_rescued_data` column instead

Explanation

Correct answer:

The records are kept with quantity set to NULL, because the original mismatched value is captured in the `_rescued_data` column instead

Why this is correct:

With an explicit schema and the default rescued-data column enabled, a JSON field that cannot be parsed as the declared INT is not silently converted into the wrong type. The row is retained, quantity is NULL, and the original mismatched value is serialized into `_rescued_data` for later inspection or repair.

Why the other options are wrong:

The schema evolves quantity to STRING, and the stream restarts so that later records keep their text values in the quantity column as they arrive: Schema evolution for new columns is a separate behavior; a value mismatch does not automatically widen this explicitly declared INT column to STRING. The rescued column captures the incompatible value instead.

The stream fails on the first bad value, with the explicitly supplied schema turning each type mismatch into a hard parsing error: A type mismatch is rescued rather than treated as a fatal JSON parsing error when the rescued-data column is enabled. Malformed JSON syntax is a different failure case.

The records are dropped from the output, and Auto Loader writes the rejected values to the stream's checkpoint directory for later review: The rescued-data column is designed to keep the record and preserve the incompatible field content. The checkpoint tracks stream progress; it is not a quarantine location for rejected row values.

References:

1. Configure schema inference and evolution in Auto Loader | Databricks on AWS:

<https://docs.databricks.com/aws/en/ingestion/cloud-object-storage/auto-loader/schema>

Question 3

The Delta table `ops.silver.deliveries` holds three years of rows and gets hourly appends. Both `order_ts` and `delivered_ts` are non-null, and nobody has checked whether older rows satisfy the rule below. Appends are paused for tonight's maintenance window, and the team wants the rule in place before they resume:

```
ALTER TABLE ops.silver.deliveries
ADD CONSTRAINT delivered_after_order CHECK (delivered_ts >= order_ts);
```

Which plan achieves that?

- A.** Add the constraint as NOT VALID tonight, as other SQL databases allow, and then run `VALIDATE CONSTRAINT` on the stored rows after the window
- B.** Run the statement in the window, since Databricks adds the constraint and then flags any stored rows that break the rule so they can be corrected later
- C.** Run the statement as soon as appends pause, since it checks only rows written after the constraint exists and leaves the stored rows untouched
- D.** Query the table for rows where `delivered_ts < order_ts`, correct or quarantine them, then run the statement, since it checks existing data first

Correct answer: D

D. Query the table for rows where `delivered_ts < order_ts`, correct or quarantine them, then run the statement, since it checks existing data first

Explanation

Correct answer

Query the table for rows where `delivered_ts < order_ts`, correct or quarantine them, then run the statement, since it checks existing data first

Why this is correct

`ALTER TABLE ADD CONSTRAINT` verifies that every existing row satisfies the `CHECK` before adding it, and if any row fails, the statement fails and no constraint is added. Since nobody has checked three years of history, the reliable plan is to find violators with the inverse predicate (`delivered_ts < order_ts`), fix them or move them to a quarantine table, and then run the `ALTER`. With appends paused, no new bad row can land in between, and once the constraint exists, violating writes are rejected.

Why the other options are wrong

Run the statement in the window, since Databricks adds the constraint and then flags any stored rows that break the rule so they can be corrected later - A single violating stored row makes the whole `ALTER` fail, and nothing is added. There is no partial success with a list of rows to clean up

afterward, so the table would still lack the rule when appends resume.

Add the constraint as NOT VALID tonight, as other SQL databases allow, and then run VALIDATE CONSTRAINT on the stored rows after the window - Deferring validation with NOT VALID and a later VALIDATE CONSTRAINT is PostgreSQL behaviour. The Databricks ADD CONSTRAINT clause offers no deferred mode; the existing-row check runs as part of the statement itself.

Run the statement as soon as appends pause, since it checks only rows written after the constraint exists and leaves the stored rows untouched - Pausing appends only prevents new bad rows. Validation also covers the three years already stored, so any violator in that history still makes the statement fail.

References

- [Constraints on Databricks](#)

- [ADD CONSTRAINT clause](#)

Question 4

The `finance_leads` group owns the schema `corp.finance`. A table in it, `corp.finance.ledger`, is owned by a different service principal. `finance_leads` must grant `SELECT` on `ledger` to auditors, and nobody holds `MANAGE`.

Can `finance_leads` grant it?

- A. No, because only the table owner or a metastore admin can grant privileges on a table, whoever happens to own its parent schema
- B. Yes, because the owner of the schema that contains a table can grant privileges on that table without having to own the table itself
- C. No, because the group first needs `ALL PRIVILEGES` on the table, which would include the ability to grant `SELECT` to other principals
- D. Yes, but only after the service principal transfers ownership of `ledger` to the group, which can then grant `SELECT` on the table itself

Correct answer: B

B. Yes, because the owner of the schema that contains a table can grant privileges on that table without having to own the table itself

Explanation

Correct answer:

Yes, because the owner of the schema that contains a table can grant privileges on that table without having to own the table itself

Why this is correct:

Unity Catalog permits the owner of a catalog or schema containing an object to grant privileges on that child object, even when another principal owns the table. Therefore, the `finance_leads` schema owner can grant auditors `SELECT` without table ownership or `MANAGE`.

Why the other options are wrong:

Yes, but only after the service principal transfers ownership of `ledger` to the group, which can then grant `SELECT` on the table itself: Ownership transfer is unnecessary because the schema owner already has authority to grant privileges on contained objects. Requiring a transfer would add an operation the privilege model does not require.

No, because only the table owner or a metastore admin can grant privileges on a table, whoever happens to own its parent schema: The table owner is not the only grantor recognized by Unity Catalog. The owner of the containing schema can also administer grants on the child table.

No, because the group first needs `ALL PRIVILEGES` on the table, which would include the ability

to grant SELECT to other principals: ALL PRIVILEGES is much broader than the requested SELECT grant, and table privileges do not need to be escalated just to issue a grant. The schema owner's grant authority is sufficient here.

References:

1. Manage privileges in Unity Catalog | Databricks on AWS:

<https://docs.databricks.com/aws/en/data-governance/unity-catalog/manage-privileges/ownership>

Question 5

An engineer edited a notebook in a Databricks Git folder. Its revision history shows several saved revisions, but nothing has been committed or pushed from the Git folder yet.

Are those edits available to a reviewer in a pull request on the remote Git provider?

- A. No, because revision history stays in the workspace; the edits reach the remote only after they are committed and pushed from the Git folder
- B. Yes, because saving a notebook inside a Git folder stages the change, and the Git provider shows staged changes in the pull request
- C. No, because they must be committed first; once committed in the Git folder, the edits appear in the pull request without a separate push
- D. No, because notebooks in Git folders are stored as .dbc archives, which the Git provider cannot display in a pull request for review

Correct answer: A

A. No, because revision history stays in the workspace; the edits reach the remote only after they are committed and pushed from the Git folder

Explanation

Correct answer:

No, because revision history stays in the workspace; the edits reach the remote only after they are committed and pushed from the Git folder

Why this is correct:

Revision history inside a Databricks Git folder is workspace-side editing history; it does not publish commits to the remote provider. The reviewer sees the change in the pull request only after the Git-folder changes are committed and pushed to the remote branch.

Why the other options are wrong:

Yes, because saving a notebook inside a Git folder stages the change, and the Git provider shows staged changes in the pull request: Saving an edit does not stage it for a remote pull request. Git staging and committing are explicit version-control operations, and the remote also needs a push.

No, because they must be committed first; once committed in the Git folder, the edits appear in the pull request without a separate push: A local commit records the change in the Git folder's local branch but does not transmit it to the Git provider. Push is the step that updates the remote branch used by the pull request.

No, because notebooks in Git folders are stored as .dbc archives, which the Git provider cannot display in a pull request for review: The blocker is that edits remain local, not that notebook files are

inherently unreadable to Git providers. Once committed and pushed, notebook changes can be reviewed in the pull request.

References:

1. Databricks Git folders | Databricks on AWS:

<https://docs.databricks.com/aws/en/repos/git-operations-with-repos>

Question 6

An Auto Loader stream starts for the first time with a new checkpoint. The landing directory already holds three years of history loaded elsewhere, and only files landing after the first start may be processed.

Which reader option should be set before the first start?

- A. `cloudFiles.includeExistingFiles` set to false, so that the stream's first run ignores the files already present in the directory
- B. `startingOffsets` set to latest, so the stream begins at the newest position in the directory and passes over the files already there
- C. `cloudFiles.backfillInterval` set to 0, so the stream skips its initial listing and processes only the files that arrive afterward
- D. `cloudFiles.useNotifications` set to true, so Auto Loader discovers only files whose arrival events are published after its first start

Correct answer: A

A. `cloudFiles.includeExistingFiles` set to false, so that the stream's first run ignores the files already present in the directory

Explanation

Correct answer:

`cloudFiles.includeExistingFiles` set to false, so that the stream's first run ignores the files already present in the directory

Why this is correct:

`cloudFiles.includeExistingFiles=false` tells Auto Loader not to ingest files already present when the stream starts for the first time. Set it before initializing the new checkpoint, because the checkpoint records the stream's discovery progress for later restarts.

Why the other options are wrong:

`cloudFiles.backfillInterval` set to 0, so the stream skips its initial listing and processes only the files that arrive afterward: Backfill settings control how Auto Loader performs backfill processing; setting an interval to zero is not the documented first-start switch for excluding the directory's existing files.

`cloudFiles.useNotifications` set to true, so Auto Loader discovers only files whose arrival events are published after its first start: Notifications change how new files are discovered, not whether files already in the location are included at initial startup. Event delivery does not imply post-start-only semantics.

startingOffsets set to latest, so the stream begins at the newest position in the directory and passes over the files already there: startingOffsets configures Kafka source offsets, not file positions in an Auto Loader directory. Auto Loader uses its own initial-file option and checkpoint.

References:

1. Spark API options reference | Databricks on AWS:

<https://docs.databricks.com/aws/en/spark/api-options>

Question 7

A bundle keeps each job definition in its own YAML file under `resources/`, and `databricks.yml` does not list them yet. Deployments currently create none of those jobs.

What should `databricks.yml` add?

- A. A targets mapping that lists the `resources/` path under each target, so every environment deploys the jobs defined in those files
- B. An artifacts mapping for `resources/`, so the bundle builds those YAML files into job definitions before the deploy step begins
- C. A sync mapping with `resources/` under its include list, so those files upload to the workspace and each one becomes a deployed job
- D. An include mapping with a pattern such as `resources/*.yml`, so those files are merged into the bundle configuration on each deploy

Correct answer: D

D. An include mapping with a pattern such as `resources/*.yml`, so those files are merged into the bundle configuration on each deploy

Explanation

Correct answer:

An include mapping with a pattern such as `resources/*.yml`, so those files are merged into the bundle configuration on each deploy

Why this is correct:

Bundle configuration files other than the main `databricks.yml` must be listed in its top-level include sequence. A glob such as `resources/*.yml` incorporates those resource definitions into the configuration that bundle validation and deployment process.

Why the other options are wrong:

A targets mapping that lists the `resources/` path under each target, so every environment deploys the jobs defined in those files: Targets select deployment-specific settings and overrides; they do not discover configuration files on disk. Resource definitions must be included at the bundle configuration level.

An artifacts mapping for `resources/`, so the bundle builds those YAML files into job definitions before the deploy step begins: artifacts describes build outputs such as wheels, not job-resource configuration YAML. Defining a bundle resource is not a build-artifact packaging step.

A sync mapping with `resources/` under its include list, so those files upload to the workspace and each one becomes a deployed job: sync controls which project files are synchronized to the

workspace. Uploading a YAML file does not merge its resource definition into the bundle deploy graph.

References:

1. Configuration reference | Databricks on AWS:

<https://docs.databricks.com/aws/en/dev-tools/bundles/reference>

Question 8

A **TIMESTAMP** column `local_time` holds Los Angeles wall-clock readings that were parsed without an offset under a UTC session, so they have not been shifted. Readings span summer and winter but avoid clock-change gaps.

Which expression converts the readings to UTC?

- A. `from_utc_timestamp(local_time, 'America/Los_Angeles')`, which treats each value as UTC time and shifts it forward into Los Angeles local time
- B. `to_utc_timestamp(local_time, 'America/Los_Angeles')`, which treats each value as Los Angeles time and applies the offset in force on its date
- C. `to_utc_timestamp(local_time, 'UTC')`, which treats each value as UTC time already and therefore leaves the stored readings exactly as they are
- D. `local_time - INTERVAL 8 HOURS`, which applies the standard Pacific offset to each reading in the column, whatever the date of that reading

Correct answer: B

B. `to_utc_timestamp(local_time, 'America/Los_Angeles')`, which treats each value as Los Angeles time and applies the offset in force on its date

Explanation

Correct answer:

`to_utc_timestamp(local_time, 'America/Los_Angeles')`, which treats each value as Los Angeles time and applies the offset in force on its date

Why this is correct:

`to_utc_timestamp` interprets a timestamp value as wall-clock time in the supplied zone and converts it to the equivalent UTC instant. Passing the region-based `America/Los_Angeles` zone lets the applicable Pacific offset vary between summer and winter, unlike a fixed-hour subtraction.

Why the other options are wrong:

`to_utc_timestamp(local_time, 'UTC')`, which treats each value as UTC time already and therefore leaves the stored readings exactly as they are: Passing UTC tells the function the values already represent UTC wall-clock time, so it does not correct their Los Angeles interpretation. The source zone must describe how the input readings were recorded.

`from_utc_timestamp(local_time, 'America/Los_Angeles')`, which treats each value as UTC time and shifts it forward into Los Angeles local time: `from_utc_timestamp` performs the opposite conversion: it takes a UTC instant and renders it in the requested local zone. Here the source is local time and the desired result is UTC.

local_time: INTERVAL 8 HOURS, which applies the standard Pacific offset to each reading in the column, whatever the date of that reading - Subtracting eight hours hard-codes standard time and ignores the seven-hour daylight offset. A named time zone applies the date-specific offset rules.

References:

1. to_utc_timestamp function | Databricks on AWS:

https://docs.databricks.com/aws/en/sql/language-manual/functions/to_utc_timestamp

Question 9

An analyst has a 2 MB CSV file on a laptop and needs it as a Unity Catalog managed table once, for a quick comparison. She holds **CREATE TABLE** on a schema but no volume exists, and she lacks **CREATE VOLUME**.

Which approach fits best?

- A. A **COPY INTO** statement that names the laptop's local folder, which loads the file once and records it so reruns skip it
- B. The Create or modify table upload page, which previews the file, lets her confirm the schema, and writes a managed table
- C. The workspace file browser, which stores the file next to her notebooks so that she can query it by path as a governed table
- D. Upload the file into a new Unity Catalog volume, then create the table from it with **CREATE TABLE AS SELECT** over `read_files`

Correct answer: B

B. The Create or modify table upload page, which previews the file, lets her confirm the schema, and writes a managed table

Explanation

Correct answer:

The Create or modify table upload page, which previews the file, lets her confirm the schema, and writes a managed table

Why this is correct:

The workspace's create-table-from-file workflow can upload a small local CSV, preview and adjust inferred schema, and create a Unity Catalog managed table directly. It meets the one-time import need without first creating a volume or staging cloud files.

Why the other options are wrong:

The workspace file browser, which stores the file next to her notebooks so that she can query it by path as a governed table: Workspace files are for notebooks and project assets, not automatically queryable as governed Unity Catalog table data. The upload must use a supported table-creation or storage workflow.

Upload the file into a new Unity Catalog volume, then create the table from it with **CREATE TABLE AS SELECT over `read_files`:** Creating the proposed volume requires **CREATE VOLUME**, which she does not have. The schema-level **CREATE TABLE** privilege is enough for the direct create-from-file workflow.

A COPY INTO statement that names the laptop's local folder, which loads the file once and records it so reruns skip it: COPY INTO reads files from an accessible cloud storage or volume path; it cannot read a path on the analyst's laptop. First-stage upload is required for that command.

References:

1. Create or modify a table using file upload | Databricks on AWS:

<https://docs.databricks.com/aws/en/ingestion/create-or-modify-table>

Question 10

A managed Delta table used liquid clustering on `order_date`. The team runs `ALTER TABLE ... CLUSTER BY (customer_id, region)` because query patterns changed, and existing files should now follow the new keys.

What should the team run next?

- A. `VACUUM` on the table, which removes the files clustered on `order_date` so that later reads use the new clustering keys alone
- B. A plain `OPTIMIZE` on the table, which rewrites each existing file with the new clustering keys the first time it runs afterward
- C. `OPTIMIZE` with the `FULL` option, which reclusters the existing records too instead of only the data that is not yet clustered
- D. Nothing further, because `ALTER TABLE CLUSTER BY` rewrites each existing data file with the new clustering keys straight away

Correct answer: C

C. `OPTIMIZE` with the `FULL` option, which reclusters the existing records too instead of only the data that is not yet clustered

Explanation

Correct answer:

`OPTIMIZE` with the `FULL` option, which reclusters the existing records too instead of only the data that is not yet clustered

Why this is correct:

Changing liquid-clustering keys changes how future writes and incremental optimization organize data, but it does not rewrite existing files. `OPTIMIZE FULL` forces a recluster of the existing records using the new keys so the stored layout catches up with the metadata change.

Why the other options are wrong:

Nothing further, because `ALTER TABLE CLUSTER BY` rewrites each existing data file with the new clustering keys straight away: Changing the metadata affects future writes and optimization; it does not immediately rewrite prior files. Existing rows need an explicit full recluster.

A plain `OPTIMIZE` on the table, which rewrites each existing file with the new clustering keys the first time it runs afterward: The standard optimization can leave eligible-looking legacy files untouched under its incremental strategy. `FULL` forces recluster of existing data using the new keys.

`VACUUM` on the table, which removes the files clustered on `order_date` so that later reads use the new clustering keys alone: `VACUUM` removes obsolete unreferenced files after retention; it does not

reorganize active rows by clustering keys. Reclaiming old storage is different from recluster table data.

References:

1. Use liquid clustering for tables | Databricks on AWS:

<https://docs.databricks.com/aws/en/delta/clustering>

Question 11

A dashboard job must run whenever corp.gold.daily_sales, a materialized view, finishes a refresh. The view refreshes on its own schedule, which another team changes without notice.

Which trigger should the dashboard job use?

- A.** A continuous trigger on the dashboard job, which keeps a run active that reads the view each time the other team refreshes it
- B.** A file arrival trigger on the view's storage location, which starts the job when the refresh writes new data files to that path
- C.** A table update trigger on the materialized view, which starts the job after each refresh that the view commits to its table
- D.** A scheduled trigger every 15 minutes that checks the view's last refresh time and exits early when nothing has changed since

Correct answer: C

C. A table update trigger on the materialized view, which starts the job after each refresh that the view commits to its table

Explanation

Correct answer:

A table update trigger on the materialized view, which starts the job after each refresh that the view commits to its table

Why this is correct:

A table update trigger watches the data object and launches the job when that source table is updated, so the dashboard task need not know or duplicate the materialized view's schedule. Since a refreshed materialized view updates its stored results, the trigger follows the actual refresh event.

Why the other options are wrong:

A file arrival trigger on the view's storage location, which starts the job when the refresh writes new data files to that path: Watching a storage location ties the run to file arrivals, not completion of the materialized view refresh. Internal file rewrites are an implementation detail; the requirement is table-level notification.

A continuous trigger on the dashboard job, which keeps a run active that reads the view each time the other team refreshes it: Nothing in this trigger links the dashboard job to the materialized view's refresh event; it schedules the dashboard job's own repeated runs instead.

A scheduled trigger every 15 minutes that checks the view's last refresh time and exits early when nothing has changed since: Polling every 15 minutes introduces needless runs and can delay

notification, while the table-update trigger can respond directly to the source update.

References:

1. Trigger jobs when source tables are updated | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/jobs/trigger-table-update>

Question 12

A classic cluster fails during startup after an admin adds a cluster-scoped init script. The event log shows an init script failure, cluster log delivery is configured, and the same cluster started normally before the change.

What should the engineer check first?

- A. The driver's log4j output from that start, which captures each init script's stdout and stderr along with the Spark startup messages
- B. The init script logs from that start in the cluster log destination, which capture the script's stdout, stderr, and failing command
- C. The Spark UI from that start, whose Executors tab shows the init script's output for each node along with the exit code it returned
- D. The workspace audit log for that start, which records each init script's output and the command that returned a nonzero exit code

Correct answer: B

B. The init script logs from that start in the cluster log destination, which capture the script's stdout, stderr, and failing command

Explanation

Correct answer:

The init script logs from that start in the cluster log destination, which capture the script's stdout, stderr, and failing command

Why this is correct:

When log delivery is configured, init-script stdout and stderr are written under the cluster's delivered init_scripts log directory. Since the startup event identifies an init-script failure, checking that run's script logs is the shortest path to its command output and error.

Why the other options are wrong:

The workspace audit log for that start, which records each init script's output and the command that returned a nonzero exit code: Audit logs record administrative changes such as global init-script edits, not the per-run stdout/stderr needed to diagnose this cluster-scoped script failure.

The Spark UI from that start, whose Executors tab shows the init script's output for each node along with the exit code it returned: The Spark UI is available after Spark has started and focuses on jobs, stages, and executors. The failing initialization occurs during cluster startup before that UI can expose the script's output.

The driver's log4j output from that start, which captures each init script's stdout and stderr along

with the Spark startup messages: Driver application logs are not the dedicated location for init-script stdout and stderr. The script logs separate those messages and can identify the failing shell command.

References:

1. Init script logging | Databricks on AWS: <https://docs.databricks.com/aws/en/init-scripts/logs>

Question 13

A fleet platform publishes telemetry to a Kafka topic. The team must land events in a bronze Delta table within seconds, no files touch cloud storage, and each micro-batch runs custom foreachBatch logic the team writes.

Which ingestion approach should the team choose?

- A.** A Lakeflow Connect managed connector for Kafka, which lands the topic in the bronze table and runs the team's foreachBatch logic on each batch
- B.** A scheduled COPY INTO job every minute, which reads the topic's latest messages and skips those already loaded through its load history
- C.** A Structured Streaming query that reads the topic with the kafka source and writes to the bronze table with a checkpoint for its offsets
- D.** Auto Loader pointed at the Kafka brokers, which discovers new messages on the topic the same way that it discovers new files in storage

Correct answer: C

C. A Structured Streaming query that reads the topic with the kafka source and writes to the bronze table with a checkpoint for its offsets

Explanation

Correct answer:

A Structured Streaming query that reads the topic with the kafka source and writes to the bronze table with a checkpoint for its offsets

Why this is correct:

Spark Structured Streaming has a Kafka source that continuously reads topic records and tracks progress in checkpointed offsets. The custom foreachBatch callback runs on each micro-batch, giving the team its own batch processing logic before each batch is written to the bronze Delta table.

Why the other options are wrong:

Auto Loader pointed at the Kafka brokers, which discovers new messages on the topic the same way that it discovers new files in storage: Auto Loader is a file-ingestion source for cloud storage; it discovers files rather than connecting directly to Kafka brokers. Kafka offsets and topic partitions require the Kafka source.

A Lakeflow Connect managed connector for Kafka, which lands the topic in the bronze table and runs the team's foreachBatch logic on each batch: A managed connector controls its supported ingestion flow and does not run arbitrary user-authored foreachBatch code per micro-batch. This requirement calls for a directly controlled streaming query.

A scheduled COPY INTO job every minute, which reads the topic's latest messages and skips those already loaded through its load history: COPY INTO loads files from storage, not Kafka messages. A scheduled file-copy command cannot provide the required seconds-level topic consumption or custom micro-batch callback.

References:

1. Structured Streaming + Kafka integration guide | Apache Spark:

<https://spark.apache.org/docs/3.5.6/structured-streaming-kafka-integration.html>

Question 14

A query must list only customers whose total order amount exceeds 10,000, one row per customer with that total. The orders table holds one row per order with customer_id and amount.

Which query meets this requirement?

- A. Sum amount grouped by customer_id, with QUALIFY SUM(amount) > 10000 after the GROUP BY clause to filter groups once their totals exist
- B. Sum amount grouped by customer_id, with WHERE amount > 10000 so that only large individual orders count toward each customer's total
- C. Sum amount grouped by customer_id, with WHERE SUM(amount) > 10000 placed before the GROUP BY clause to filter out smaller customers
- D. Sum amount grouped by customer_id, with HAVING SUM(amount) > 10000 after the GROUP BY clause to filter out smaller customer groups

Correct answer: D

D. Sum amount grouped by customer_id, with HAVING SUM(amount) > 10000 after the GROUP BY clause to filter out smaller customer groups

Explanation

Correct answer:

Sum amount grouped by customer_id, with HAVING SUM(amount) > 10000 after the GROUP BY clause to filter out smaller customer groups

Why this is correct:

GROUP BY customer_id first forms one group per customer and computes the aggregate total for that group. HAVING filters groups using aggregate results, so SUM(amount) > 10000 retains only customers whose total exceeds the threshold.

Why the other options are wrong:

Sum amount grouped by customer_id, with QUALIFY SUM(amount) > 10000 after the GROUP BY clause to filter groups once their totals exist: QUALIFY filters results of window functions after window evaluation; this query needs a group-level aggregate filter and contains no window calculation. Use HAVING for the aggregate condition.

Sum amount grouped by customer_id, with WHERE SUM(amount) > 10000 placed before the GROUP BY clause to filter out smaller customers: WHERE filters input rows before grouping, so it cannot directly test the aggregate SUM(amount). The threshold applies to the completed group, after aggregation.

Sum amount grouped by customer_id, with WHERE amount > 10000 so that only large individual

orders count toward each customer's total: Filtering individual orders above 10,000 changes which rows contribute to each total. A customer with many smaller orders could exceed the threshold, so compare the sum after grouping instead.

References:

1. HAVING clause | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/pt/sql/language-manual/sql-ref-syntax-qry-select-having>

Question 15

A daily job appends rows to a Delta table. Today's source adds a new column, region, that the table schema does not include. No table property or session setting enables automatic schema changes.

What happens to the append, and how can it keep the new column?

- A. The write succeeds and adds region on its own, because Delta evolves a table's schema whenever an append brings in a column it has not seen
- B. The write succeeds and quietly drops region, with Delta keeping the columns already defined in the table schema and ignoring extra ones
- C. The write fails on the schema mismatch; setting the mergeSchema option on this write lets Delta add region to the table schema as part of the append
- D. The write fails on the schema mismatch; setting overwriteSchema on this append lets Delta add region while keeping the table's existing rows

Correct answer: C

C. The write fails on the schema mismatch; setting the mergeSchema option on this write lets Delta add region to the table schema as part of the append

Explanation

Correct answer:

The write fails on the schema mismatch; setting the mergeSchema option on this write lets Delta add region to the table schema as part of the append

Why this is correct:

Delta validates incoming columns against the target schema and rejects an append that introduces region when schema evolution is not enabled. The per-write mergeSchema option authorizes adding that source column to the existing schema while the operation remains an append, preserving existing table rows.

Why the other options are wrong:

The write succeeds and quietly drops region, with Delta keeping the columns already defined in the table schema and ignoring extra ones: Delta schema enforcement does not silently discard an unexpected source column under the stated settings. The mismatch is reported so data is not quietly lost.

The write fails on the schema mismatch; setting overwriteSchema on this append lets Delta add region while keeping the table's existing rows: overwriteSchema replaces the schema during an overwrite operation; it is not the append-time schema-evolution control and could replace table data if used with overwrite mode.

The write succeeds and adds region on its own, because Delta evolves a table's schema whenever an append brings in a column it has not seen: Delta does not infer permission to evolve from the mere presence of an extra field. Evolution must be enabled for the write or configured separately.

References:

1. Schema enforcement | Databricks on AWS:

<https://docs.databricks.com/aws/en/tables/schema-enforcement>

Question 16

CSV files hold `order_id`, `amount_cents`, and a `loaded_by` column that must not be stored. The target table needs `order_id` and amount in dollars, converted during the load in one statement, with COPY INTO's file tracking kept.

Which approach meets this?

- A. COPY INTO with FROM (SELECT `order_id`, `amount_cents` / 100 AS `amount` FROM '<path>'), which selects and converts the columns during the load
- B. INSERT INTO the table from a SELECT over `read_files` with the same conversion, which transforms the columns and tracks the files it loaded
- C. COPY INTO with FROM '<path>' and FORMAT_OPTIONS ('columns' = '`order_id`, `amount`'), which renames and converts the columns during the load
- D. COPY INTO a staging table from '<path>', then INSERT the converted columns into the target, which keeps file tracking for the load

Correct answer: A

A. COPY INTO with FROM (SELECT `order_id`, `amount_cents` / 100 AS `amount` FROM '<path>'), which selects and converts the columns during the load

Explanation

Correct answer:

COPY INTO with FROM (SELECT `order_id`, `amount_cents` / 100 AS `amount` FROM '<path>'), which selects and converts the columns during the load

Why this is correct:

COPY INTO ... FROM (SELECT ...) can project chosen source fields and apply expressions before writing the Delta target. This single command omits `loaded_by`, divides cents into dollars, and preserves COPY INTO's file tracking and idempotent loading behavior.

Why the other options are wrong:

INSERT INTO the table from a SELECT over `read_files` with the same conversion, which transforms the columns and tracks the files it loaded: `read_files` with a normal insert can transform the values, but it does not use COPY INTO's per-file load history and retry behavior. That tracking requirement makes the COPY INTO source query significant.

COPY INTO with FROM '<path>' and FORMAT_OPTIONS ('columns' = '`order_id`, `amount`'), which renames and converts the columns during the load: FORMAT_OPTIONS configure how the file reader parses its format; they do not rename columns or apply arithmetic expressions. Use the source SELECT list for projection and conversion.

COPY INTO a staging table from '<path>', then INSERT the converted columns into the target, which keeps file tracking for the load: A staging-table workflow requires multiple statements and a separate insert, rather than performing the requested projection and conversion in one COPY INTO load. The file tracker belongs to that ingestion command.

References:

1. COPY INTO | Databricks on AWS:

<https://docs.databricks.com/aws/en/sql/language-manual/delta-copy-into>

Question 17

A job's recent runs take 50 minutes instead of the usual 30. It has six tasks, and the engineer wants to see which task's duration grew across the last few weeks of runs without querying any system tables.

Where should the engineer look?

- A. The job's runs list, which shows each run's start time, total duration, and final status across all of the recent runs of the job
- B. The latest run's timeline view, which shows each task's start time and duration, so the task that grew over the recent weeks stands out
- C. The Spark UI of the latest run, whose stage timeline also shows the durations of the same job's tasks across its earlier runs
- D. The job's matrix view, whose bars show each run's duration and each task's duration side by side across the recent runs of the job

Correct answer: D

D. The job's matrix view, whose bars show each run's duration and each task's duration side by side across the recent runs of the job

Explanation

Correct answer:

The job's matrix view, whose bars show each run's duration and each task's duration side by side across the recent runs of the job

Why this is correct:

The job's matrix view lays out recent runs against tasks, and each run/task bar's height represents its duration. Comparing the bars for each task across runs makes a multiweek increase visible without querying system tables.

Why the other options are wrong:

The Spark UI of the latest run, whose stage timeline also shows the durations of the same job's tasks across its earlier runs: The Spark UI analyzes Spark applications and stages for a selected run, not the history of job task durations across weeks. Use the job-level history view for that comparison.

The latest run's timeline view, which shows each task's start time and duration, so the task that grew over the recent weeks stands out: A single-run timeline can explain ordering and overlap for one execution, but it cannot show the multiweek change across previous executions.

The job's runs list, which shows each run's start time, total duration, and final status across all of the recent runs of the job: Overall run duration combines all six tasks, so it cannot identify which

individual task grew. The matrix exposes task-level bars across runs.

References:

1. **Monitor Lakeflow Jobs | Databricks on AWS:** <https://docs.databricks.com/aws/en/jobs/monitor>

Question 18

A Lakeflow Connect Salesforce pipeline runs incrementally with history tracking off. Salesforce admins permanently hard-delete several accounts, bypassing the recycle bin, and those accounts must disappear from the destination table.

What must the team do?

- A. Turn on history tracking, because SCD type 2 marks hard-deleted accounts as inactive by filling in their `__END_AT` column values
- B. Delete the matching rows from the destination table by hand, because the connector does not allow full refreshes of tables it manages
- C. Nothing further, because the next incremental update detects hard-deleted records and removes them from the destination table
- D. Run a full refresh of the destination table, because incremental updates do not pick up records that were hard-deleted in Salesforce

Correct answer: D

D. Run a full refresh of the destination table, because incremental updates do not pick up records that were hard-deleted in Salesforce

Explanation

Correct answer:

Run a full refresh of the destination table, because incremental updates do not pick up records that were hard-deleted in Salesforce

Why this is correct:

The Salesforce connector does not automatically detect hard deletes that bypass the recycle bin during incremental ingestion. A full refresh clears and rebuilds the selected destination from the current source, so records that no longer exist in Salesforce disappear from the target.

Why the other options are wrong:

Delete the matching rows from the destination table by hand, because the connector does not allow full refreshes of tables it manages: Manual destination edits create drift and are not required for recovery. The connector supports a full refresh that reconciles target contents to the source snapshot.

Turn on history tracking, because SCD type 2 marks hard-deleted accounts as inactive by filling in their `__END_AT` column values: History tracking models captured changes as SCD Type 2 history; it cannot create a deletion event the incremental source feed never delivered. A full refresh is needed to discover hard-deleted records.

Nothing further, because the next incremental update detects hard-deleted records and removes

them from the destination table: Because the hard deletion bypassed the recycle bin, it is not surfaced through the ordinary incremental deletion mechanism. Later incremental runs therefore do not automatically remove that row.

References:

1. Salesforce ingestion connector FAQs | Databricks on AWS:

<https://docs.databricks.com/aws/en/ingestion/lakeflow-connect/salesforce-faq>

Question 19

A COPY INTO load reads from /Volumes/corp/landing/events/, which holds CSV files in dated subfolders plus a few JSON manifest files. Only the CSV files from 2026 subfolders such as 2026-09-01/ may be loaded.

```
COPY INTO corp.bronze.events  
FROM '/Volumes/corp/landing/events/'  
FILEFORMAT = CSV
```

Which clause completes the command?

- A. FILES = ('2026-*/*.csv'), which limits the load to the CSV files that sit inside the 2026 subfolders of the source path
- B. COPY_OPTIONS ('pattern' = '*.csv'), which limits the load to the CSV files that sit inside the 2026 subfolders of the path
- C. PATTERN = '2026-*/*.csv', which limits the load to the CSV files that sit inside the 2026 subfolders of the source path
- D. FORMAT_OPTIONS ('pathGlobFilter' = '2026'), which limits the load to the files in subfolders whose names start with 2026

Correct answer: C

C. PATTERN = '2026-*/*.csv', which limits the load to the CSV files that sit inside the 2026 subfolders of the source path

Explanation

Correct answer:

PATTERN = '2026-*/*.csv', which limits the load to the CSV files that sit inside the 2026 subfolders of the source path

Why this is correct:

PATTERN filters source files by a glob-style path pattern relative to the input location. 2026-*/*.csv selects CSV files one directory below the root when that directory name begins with 2026, excluding the JSON manifests and other years.

Why the other options are wrong:

FORMAT_OPTIONS ('pathGlobFilter' = '2026'), which limits the load to the files in subfolders whose names start with 2026: pathGlobFilter is a reader option for file name filtering, not a COPY INTO clause that selects dated subdirectories as written. This pattern must match both directory and filename components.

COPY_OPTIONS ('pattern' = '*.csv'), which limits the load to the CSV files that sit inside the 2026 subfolders of the path: COPY_OPTIONS controls copy-operation behavior rather than source path selection; pattern is not a copy option. The path filter belongs in the command's PATTERN parameter.

FILES = ('2026-*/*.csv'), which limits the load to the CSV files that sit inside the 2026 subfolders of the source path: FILES enumerates literal filenames, not a wildcard pattern. Use PATTERN when the selected files are described by a glob.

References:

1. COPY INTO | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/sql/language-manual/delta-copy-into>

Question 20

A team is moving Parquet files on cloud storage to a lakehouse. It needs atomic commits with a transaction log for time travel, and central access control over tables across several workspaces.

Which TWO components provide these capabilities?

- A. Delta Lake, which adds a transaction log and ACID commits on top of the Parquet data files stored in cloud object storage
- B. The workspace Hive metastore, which governs table permissions centrally for every workspace that reads the same storage
- C. Delta Sharing, which governs table access centrally for every workspace in the account through its shares and recipients
- D. Unity Catalog, which governs access to tables centrally for all of the workspaces that are attached to its metastore
- E. Structured Streaming checkpoints, which record every commit so that readers can time travel to earlier table versions

Correct answers: A, D

- A. Delta Lake, which adds a transaction log and ACID commits on top of the Parquet data files stored in cloud object storage
- D. Unity Catalog, which governs access to tables centrally for all of the workspaces that are attached to its metastore

Explanation

Correct answer:

Delta Lake, which adds a transaction log and ACID commits on top of the Parquet data files stored in cloud object storage

Unity Catalog, which governs access to tables centrally for all of the workspaces that are attached to its metastore

Why this is correct:

Delta Lake adds a transaction log alongside Parquet files, enabling atomic commits and versioned snapshots for time travel. Unity Catalog supplies centralized identity, permissions, discovery, and governance for tables shared across workspaces attached to its metastore.

Why the other options are wrong:

Delta Sharing, which governs table access centrally for every workspace in the account through its shares and recipients: Delta Sharing is for securely sharing data across organizational boundaries; it does not serve as the metastore's central access-control system for all attached workspaces.

The workspace Hive metastore, which governs table permissions centrally for every workspace that reads the same storage: A workspace Hive metastore does not centralize governance across several workspaces. Unity Catalog is the shared metastore-backed catalog layer that provides that scope.

Structured Streaming checkpoints, which record every commit so that readers can time travel to earlier table versions: Streaming checkpoints store progress for a streaming query, not Delta table commit history. Time travel relies on the Delta transaction log and retained data files.

References:

1. What is Delta Lake in Databricks? | Databricks on AWS: <https://docs.databricks.com/aws/en/delta>
2. What is Unity Catalog? | Databricks on AWS:
<https://docs.databricks.com/aws/en/data-governance/unity-catalog>

Question 21

Support analysts need every customer row and all ordinary attributes, but they must see a redacted email value. Compliance users must see the original email. The team wants one governed table and no additional objects for either group.

Which control fits?

- A. A row filter whose function returns email only when `is_account_group_member('compliance')` is true, so other readers receive a NULL email
- B. A dynamic view that redacts email with `is_account_group_member`, with support analysts granted on the view and compliance granted on the table
- C. A column mask on email whose function returns the unmodified address to the compliance group and a masked string to all other readers
- D. A SELECT grant on every column except email for support analysts, which hides that column from them while leaving each of the rows visible

Correct answer: C

C. A column mask on email whose function returns the unmodified address to the compliance group and a masked string to all other readers

Explanation

Correct answer:

A column mask on email whose function returns the unmodified address to the compliance group and a masked string to all other readers

Why this is correct:

A column mask transforms the value returned for one column according to the querying user's identity while leaving the row and other columns visible. A function can return the original address to compliance and a redacted value to support, all on the same governed table.

Why the other options are wrong:

A row filter whose function returns email only when `is_account_group_member('compliance')` is true, so other readers receive a NULL email: A row filter includes or excludes entire rows; it cannot keep all customer records while substituting only the email value. Column-level redaction is the mask's role.

A SELECT grant on every column except email for support analysts, which hides that column from them while leaving each of the rows visible: Unity Catalog table SELECT grants do not provide this selective per-column concealment in the described single-table setup. Withholding email entirely would also fail to return a redacted value.

A dynamic view that redacts email with is_account_group_member, with support analysts granted on the view and compliance granted on the table: A dynamic view introduces the extra governed object the stem rules out. The table-level column mask provides the caller-dependent result directly on the existing table.

References:

1. Row filters and column masks | Databricks on AWS:

<https://docs.databricks.com/aws/en/data-governance/unity-catalog/filters-and-masks>

Question 22

A nightly classic batch job retries interrupted tasks and has a flexible deadline. The team wants lower compute cost and accepts reclaimed workers, but reclamation should not end the run, and the cluster must still start when spot capacity is short.

Which instance purchasing setup fits?

- A. Spot instances for every worker with no on-demand fallback, while the driver stays on-demand, so a reclaimed worker is replaced and its tasks are retried
- B. Spot instances for the workers with fallback to on-demand, while the driver stays on-demand, so a reclaimed worker is replaced and its tasks are retried
- C. On-demand instances for every worker and a spot driver with fallback to on-demand, so the driver gets the discount while the workers stay protected
- D. Spot instances for the driver and every worker with fallback to on-demand, so a shortage of spot capacity at start uses on-demand machines

Correct answer: B

B. Spot instances for the workers with fallback to on-demand, while the driver stays on-demand, so a reclaimed worker is replaced and its tasks are retried

Explanation

Correct answer

Spot instances for the workers with fallback to on-demand, while the driver stays on-demand, so a reclaimed worker is replaced and its tasks are retried

Why this is correct

Spot workers reduce the variable worker cost, and fallback lets the cluster request on-demand workers when spot capacity is unavailable. Keeping the driver on-demand avoids driver reclamation; task retries provide recovery if a worker is reclaimed while the batch is running. This lowers the risk from reclamation but does not remove it, since heavy worker loss can still fail a stage.

Why the other options are wrong

On-demand instances for every worker and a spot driver with fallback to on-demand, so the driver gets the discount while the workers stay protected - This puts the discounted, reclaimable capacity on the driver, the node that coordinates the Spark application, while leaving all workers on the more expensive purchase type. It misses both the desired savings and driver protection.

Spot instances for the driver and every worker with fallback to on-demand, so a shortage of spot capacity at start uses on-demand machines - Fallback addresses a shortage while acquiring capacity;

it does not make an already-running spot driver immune to later reclamation. A reclaimed driver can terminate the Spark application.

Spot instances for every worker with no on-demand fallback, while the driver stays on-demand, so a reclaimed worker is replaced and its tasks are retried - Without fallback, the cluster can fail to start when spot capacity is unavailable. Retry logic cannot compensate for having no workers provisioned to begin the run.

References

- [Compute configuration reference | Databricks on AWS](#)

Question 23

A governed tag named pii has an approved value list. A data steward must apply pii = email to columns of corp.crm.contacts, and she currently holds no tag-related permission or privilege.

Which TWO permissions does she need?

- A. ASSIGN on the pii governed tag, which allows her to attach that tag and its allowed values to objects where she can also apply tags
- B. MODIFY on corp.crm.contacts, which covers column tags because tags are stored as part of table metadata
- C. MANAGE on the pii governed tag, which by itself allows her to attach the tag to any object in the account
- D. CREATE on governed tags, which allows her to define new pii values and attach them to the table's columns
- E. APPLY TAG on corp.crm.contacts, which allows tags to be attached to the table and each of its columns

Correct answers: A, E

- A. ASSIGN on the pii governed tag, which allows her to attach that tag and its allowed values to objects where she can also apply tags
- E. APPLY TAG on corp.crm.contacts, which allows tags to be attached to the table and each of its columns

Explanation

Correct answer:

ASSIGN on the pii governed tag, which allows her to attach that tag and its allowed values to objects where she can also apply tags

APPLY TAG on corp.crm.contacts, which allows tags to be attached to the table and each of its columns

Why this is correct:

Governed tag assignment requires both permission to use that governed tag (ASSIGN) and permission to tag the Unity Catalog object (APPLY TAG). The allowed email value is already defined, so she needs authorization at both scopes to apply pii=email to the table's columns.

Why the other options are wrong:

MODIFY on corp.crm.contacts, which covers column tags because tags are stored as part of table metadata: MODIFY changes table data, not governed-tag assignments. Tagging has its own object privilege, APPLY TAG, and the tag itself also requires ASSIGN permission.

CREATE on governed tags, which allows her to define new pii values and attach them to the table's columns: The tag and its allowed values already exist, so permission to define new tags is unnecessary. Assignment still needs the object-level APPLY TAG privilege and tag-level ASSIGN.

MANAGE on the pii governed tag, which by itself allows her to attach the tag to any object in the account: MANAGE lets its holder manage tag definitions or permissions, but it is not the specific permission to assign a governed tag value. The required tag permission is ASSIGN.

References:

1. Manage permissions on governed tags | Databricks on AWS:

<https://docs.databricks.com/aws/en/admin/governed-tags/manage-permissions>

Question 24

An orders table, aliased o, has country and service columns. A rates table, aliased r, holds exactly one price per country and service pair. The output must hold one row per order and nothing else, with a NULL price where no rate exists.

Which join meets this requirement?

- A. LEFT JOIN rates r ON o.country = r.country AND o.service = r.service, which still keeps each unmatched order and matches every order to at most one rate row
- B. LEFT JOIN rates r ON o.country = r.country with WHERE o.service = r.service, which applies the pair match while still retaining every order in the output
- C. FULL OUTER JOIN rates r ON o.country = r.country AND o.service = r.service, which keeps every order and fills in a NULL price where no rate row matches
- D. INNER JOIN rates r ON o.country = r.country AND o.service = r.service, which matches each order to at most one rate row by using both of the key columns

Correct answer: A

A. LEFT JOIN rates r ON o.country = r.country AND o.service = r.service, which still keeps each unmatched order and matches every order to at most one rate row

Explanation

Correct answer:

LEFT JOIN rates r ON o.country = r.country AND o.service = r.service, which still keeps each unmatched order and matches every order to at most one rate row

Why this is correct:

A left outer join preserves every row from the left input and supplies NULLs for right-side columns when the join predicate finds no match. Putting both country and service in ON makes the pair the matching key; the stated uniqueness of that pair in rates ensures a matching order cannot multiply.

Why the other options are wrong:

LEFT JOIN rates r ON o.country = r.country with WHERE o.service = r.service, which applies the pair match while still retaining every order in the output: The WHERE clause runs after the outer join and rejects rows whose right-side service is NULL, removing orders without a rate. Keep right-side matching predicates in ON when unmatched left rows must survive.

INNER JOIN rates r ON o.country = r.country AND o.service = r.service, which matches each order to at most one rate row by using both of the key columns: The retained-row guarantee is lost when orders have no matching rate, so this join type cannot produce the requested NULL-price rows.

FULL OUTER JOIN rates r ON o.country = r.country AND o.service = r.service, which keeps every

order and fills in a NULL price where no rate row matches: Rate-only records would become extra output rows even though the report is order-driven. Retaining unmatched data from both inputs therefore exceeds the requested left-preserving result.

References:

1. JOIN | Spark SQL documentation:

<https://spark.apache.org/docs/latest/sql-ref-syntax-qry-select-join.html>

Question 25

A Lakeflow Spark Declarative Pipeline's transformation logic changed, and every table it maintains must be recomputed from scratch once. A job already runs the pipeline through a pipeline task each night.

How should tonight's run achieve this?

- A. Enable the full refresh option on the pipeline task for this run, so the update clears and recomputes every table in the pipeline
- B. Set `pipelines.reset.allowed` to false on each table for tonight's run, so the update resets their state and recomputes them from scratch
- C. Add a notebook task before the pipeline task that runs `DROP TABLE` on each pipeline table, so tonight's update rebuilds all of them
- D. Refresh only the tables whose own logic changed, so the tables downstream of them pick up the new results through incremental updates

Correct answer: A

A. Enable the full refresh option on the pipeline task for this run, so the update clears and recomputes every table in the pipeline

Explanation

Correct answer:

Enable the full refresh option on the pipeline task for this run, so the update clears and recomputes every table in the pipeline

Why this is correct:

The pipeline task supports a full-refresh setting that requests a full refresh for the pipeline update it launches. A full refresh recomputes materialized views and clears/reprocesses streaming tables, which is the requested one-time rebuild of every maintained dataset.

Why the other options are wrong:

Set `pipelines.reset.allowed` to false on each table for tonight's run, so the update resets their state and recomputes them from scratch: The proposed configuration explicitly forbids resets, so it cannot force a recomputation. Request a full refresh on the pipeline task instead.

Add a notebook task before the pipeline task that runs `DROP TABLE` on each pipeline table, so tonight's update rebuilds all of them: Dropping pipeline-managed tables outside the pipeline interferes with the pipeline's ownership and state. Use its supported refresh operation so that the pipeline manages the rebuild consistently.

Refresh only the tables whose own logic changed, so the tables downstream of them pick up the

new results through incremental updates: Refreshing only changed tables does not guarantee every maintained table is recomputed from scratch. The requirement explicitly requests a one-time pipeline-wide refresh.

References:

1. Add tasks to jobs in Declarative Automation Bundles | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/dev-tools/bundles/job-task-types>

Question 26

A join between a 900 GB fact table and a 20 MB dimension table runs as a sort-merge join because the dimension has no statistics. Statistics cannot be collected this week, and AQE is disabled by policy.

Which change should be tested first?

- A. Add a `/*+ MERGE(dim) */` hint to the query, which asks the planner to merge the dimension table into each fact partition before the join step runs
- B. Raise `spark.sql.autoBroadcastJoinThreshold` to 50 MB, which makes the planner treat the dimension table as small enough to broadcast without statistics
- C. Add a `/*+ BROADCAST(dim) */` hint to the query, which tells the planner to send the whole dimension table to every executor before the join step runs
- D. Raise `spark.sql.shuffle.partitions` for this query, which spreads the sort-merge join across more tasks until the planner decides to broadcast the smaller side

Correct answer: C

C. Add a `/*+ BROADCAST(dim) */` hint to the query, which tells the planner to send the whole dimension table to every executor before the join step runs

Explanation

Correct answer:

Add a `/*+ BROADCAST(dim) */` hint to the query, which tells the planner to send the whole dimension table to every executor before the join step runs

Why this is correct:

The optimizer lacks size statistics for the dimension and cannot adapt the plan at runtime because AQE is disabled. A `BROADCAST(dim)` hint directly tells Spark to use the dimension as the broadcast side, avoiding the fact-table shuffle associated with the current sort-merge join; the 20 MB size is also comfortably small for replication to executors.

Why the other options are wrong:

Raise `spark.sql.shuffle.partitions` for this query, which spreads the sort-merge join across more tasks until the planner decides to broadcast the smaller side: More shuffle partitions alter parallelism, not join-strategy selection. They do not supply the missing size estimate that would make Spark independently switch to broadcast.

Add a `/*+ MERGE(dim) */` hint to the query, which asks the planner to merge the dimension table into each fact partition before the join step runs: `MERGE` asks Spark to use a sort-merge strategy, the strategy already selected. It does not copy the small relation to each executor.

Raise `spark.sql.autoBroadcastJoinThreshold` to 50 MB, which makes the planner treat the dimension table as small enough to broadcast without statistics: The automatic threshold only helps when Spark can estimate the relation's size. With no dimension statistics, changing the cutoff alone does not reliably classify it as 20 MB; the explicit hint conveys the intended strategy.

References:

- 1. Performance Tuning | Apache Spark:** <https://spark.apache.org/docs/latest/sql-performance-tuning>

Question 27

A Lakeflow Spark Declarative Pipelines dataset must discard records whose `bin_id` is null. The engineer imports pipelines as `dp` and wants the expectation named `bin_present`.

Which decorator uses the supported form?

- A. `@dp.expect("bin_present", "bin_id IS NOT NULL", "drop")`
- B. `@dp.expect_or_drop("bin_present", "bin_id IS NOT NULL")`
- C. `@dp.expect_all("bin_present", "bin_id IS NOT NULL")`
- D. `@dp.expect_or_drop({"bin_present": "bin_id IS NOT NULL"})`

Correct answer: B

B. `@dp.expect_or_drop("bin_present", "bin_id IS NOT NULL")`

Explanation

Correct answer:

`@dp.expect_or_drop("bin_present", "bin_id IS NOT NULL")`

Why this is correct:

`expect_or_drop(name, predicate)` attaches a named data-quality expectation and drops rows whose predicate evaluates false. The decorator takes the expectation name and condition as separate arguments, so this form both uses `bin_present` and removes NULL `bin_id` records.

Why the other options are wrong:

@dp.expect_all("bin_present", "bin_id IS NOT NULL"): `expect_all` accepts a mapping of expectation names to expressions and defaults to warning behavior; the positional two-string form is not its supported mapping signature and would not request row dropping.

@dp.expect("bin_present", "bin_id IS NOT NULL", "drop"): `expect` takes a name and condition but is the warning-mode decorator; the third string is not the drop-action parameter. Use the dedicated `expect_or_drop` decorator to filter violating rows.

@dp.expect_or_drop({"bin_present": "bin_id IS NOT NULL"}): The `_or_drop` decorator expects the expectation name and expression as separate arguments, not a dictionary. The dictionary form is associated with the multi-expectation APIs.

References:

1. Manage data quality with pipeline expectations | Microsoft Learn:
<https://learn.microsoft.com/en-us/azure/databricks/ldp/expectations>

Question 28

A bronze STRING column `ship_date` holds day-first values such as 31/01/2026. Silver needs a DATE column, and values that do not match the pattern should become NULL instead of failing the query. ANSI mode is on.

Which expression meets this requirement?

- A. `to_date(ship_date, 'MM/dd/yyyy')`, which parses each value with the given month-first pattern and returns NULL whenever a value does not fit it
- B. `to_date(ship_date)` with no pattern, which detects the day-first layout from the values themselves and converts them without further input
- C. `CAST(ship_date AS DATE)`, which reads the day-first text directly under ANSI mode and returns NULL for any value that it cannot interpret
- D. `try_to_date(ship_date, 'dd/MM/yyyy')`, which parses each value with the given pattern and returns NULL whenever a value does not fit it

Correct answer: D

D. `try_to_date(ship_date, 'dd/MM/yyyy')`, which parses each value with the given pattern and returns NULL whenever a value does not fit it

Explanation

Correct answer:

`try_to_date(ship_date, 'dd/MM/yyyy')`, which parses each value with the given pattern and returns NULL whenever a value does not fit it

Why this is correct:

The provided format is day/month/year, and `try_to_date` combines that explicit parser pattern with tolerant conversion. Valid dates become DATE values; invalid or nonconforming strings become NULL, which is the required behavior under ANSI mode.

Why the other options are wrong:

`to_date(ship_date, 'MM/dd/yyyy')`, which parses each value with the given month-first pattern and returns NULL whenever a value does not fit it: The format `MM/dd/yyyy` interprets the first field as a month, contrary to the day-first source. Also, ordinary `to_date` can raise on malformed values when ANSI behavior is enabled.

`CAST(ship_date AS DATE)`, which reads the day-first text directly under ANSI mode and returns NULL for any value that it cannot interpret: The implicit cast has no `dd/MM/yyyy` pattern and is not a tolerant parser. Under ANSI mode, malformed input raises instead of returning NULL.

`to_date(ship_date)` with no pattern, which detects the day-first layout from the values themselves

and converts them without further input: Without a format, Databricks expects its standard date form rather than detecting arbitrary input layouts. Day-first parsing requires the matching pattern, and invalid input must use a tolerant conversion.

References:

1. PySpark functions | Databricks on AWS:

<https://docs.databricks.com/aws/en/pyspark/reference/functions/>

Question 29

A `tickets` `DataFrame` has one row per ticket with `ticket_id`, `opened_date`, and `priority`. An engineer needs a ticket count for each priority on each date, with one output row per date and priority pair.

Which transformation produces this output?

- A. `tickets.groupBy("opened_date", "priority").count()`
- B. `tickets.groupBy("opened_date").agg(countDistinct("priority"))`
- C. `tickets.groupBy("opened_date").pivot("priority").count()`
- D. `tickets.select("opened_date", "priority").distinct().count()`

Correct answer: A

A. `tickets.groupBy("opened_date", "priority").count()`

Explanation

Correct answer:

`tickets.groupBy("opened_date", "priority").count()`

Why this is correct:

Grouping by both `opened_date` and `priority` creates one group for every observed date-priority pair, and `count()` counts the ticket rows in each group. Because `ticket_id` is unique per ticket, each row contributes once to its pair's total.

Why the other options are wrong:

`tickets.groupBy("opened_date").pivot("priority").count()`: Pivoting turns priority values into separate columns, so the output is one row per date with a wide layout. The requirement asks for one row per date-priority combination.

`tickets.groupBy("opened_date").agg(countDistinct("priority"))`: Counting distinct priorities per date returns how many priority categories occur, not how many tickets belong to each priority. The ticket rows must be counted inside each two-column group.

`tickets.select("opened_date", "priority").distinct().count()`: `distinct()` removes duplicate date-priority pairs and `.count()` then returns one overall scalar count of unique pairs. It does not produce the requested ticket count in each group.

References:

1. pyspark.sql.DataFrame.groupBy | PySpark documentation: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.groupBy.html>

Question 30

A query-based connector uses `updated_at` as its cursor column. Inserts arrive correctly, but edits leave `updated_at` unchanged, so they never reach the destination. Future edits must be captured without periodic full reloads.

Which source-side change fixes this?

- A. Make every insert and update set `updated_at` to the change time, so each edit moves the row past the cursor of the last sync
- B. Shorten the sync interval to five minutes, so the connector compares rows more often and picks up edits that kept `updated_at`
- C. Enable change data feed on the source table, so the connector reads each edit from the feed while it keeps `updated_at` as its cursor
- D. Switch the cursor to the primary key column, so the connector reads rows whose key is higher than the last sync

Correct answer: A

A. Make every insert and update set `updated_at` to the change time, so each edit moves the row past the cursor of the last sync

Explanation

Correct answer:

Make every insert and update set `updated_at` to the change time, so each edit moves the row past the cursor of the last sync

Why this is correct:

A query-based incremental connector compares the cursor value with its last-synced position to decide which records changed. Updating `updated_at` on every insert and edit makes every modified row eligible for a later incremental query, without rescanning the entire source.

Why the other options are wrong:

Shorten the sync interval to five minutes, so the connector compares rows more often and picks up edits that kept `updated_at`: Polling more often shortens detection delay but does not change the cursor comparison. If an edit leaves `updated_at` behind the last-synced value, repeated polls still miss it.

Switch the cursor to the primary key column, so the connector reads rows whose key is higher than the last sync: A primary key identifies the row but usually remains unchanged during edits, so it does not signal that a record was modified. A change-tracking cursor must advance when the row changes.

Enable change data feed on the source table, so the connector reads each edit from the feed

while it keeps updated_at as its cursor: The connector continues to use its configured cursor; enabling CDF alone does not reconfigure its ingestion mechanism. Make the cursor advance on each edit.

References:

1. Query-based connector reference | Databricks on AWS:

<https://docs.databricks.com/aws/en/ingestion/lakeflow-connect/query-based-reference>

Question 31

A job is scheduled for 09:00 every day in the America/New_York time zone. Daylight saving time begins, and nobody edits the schedule.

What happens to its local start time and the matching UTC time?

- A. It keeps its previous UTC time, so it now starts at 10:00 in New York until someone edits the schedule to follow the new offset
- B. It still starts at 09:00 in New York, because the schedule follows the named zone, so the matching UTC time moves one hour later
- C. It runs twice on the day of the change, once at 09:00 local time and once more at the UTC time that it used before the change
- D. It still starts at 09:00 in New York, because the schedule follows the named zone, so the matching UTC time moves one hour earlier

Correct answer: D

D. It still starts at 09:00 in New York, because the schedule follows the named zone, so the matching UTC time moves one hour earlier

Explanation

Correct answer:

It still starts at 09:00 in New York, because the schedule follows the named zone, so the matching UTC time moves one hour earlier

Why this is correct:

The schedule is defined in the named America/New_York time zone, so the job remains anchored to 09:00 local time through daylight-saving transitions. New York shifts from UTC-5 to UTC-4 in spring, making 09:00 local correspond to a UTC time one hour earlier than before.

Why the other options are wrong:

It still starts at 09:00 in New York, because the schedule follows the named zone, so the matching UTC time moves one hour later: The spring transition shortens New York's offset from UTC by one hour, so 09:00 local maps to 13:00 UTC instead of 14:00 UTC. The corresponding UTC time moves earlier.

It runs twice on the day of the change, once at 09:00 local time and once more at the UTC time that it used before the change: A time-zone-aware daily cron schedule adjusts its UTC execution time to preserve the specified local hour; the offset change does not create a second run at the old UTC time.

It keeps its previous UTC time, so it now starts at 10:00 in New York until someone edits the schedule to follow the new offset: Holding UTC fixed would shift the local start from 09:00 to 10:00

after the spring transition. The named zone, rather than the former numeric offset, governs the schedule.

References:

1. Run jobs on a schedule | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/jobs/scheduled>

Question 32

A bundle needs a dev target that deploys to `https://dev.example.databricks.com` under `/Workspace/Shared/bundles/orders-dev`. Authentication is configured, and the fragment sits at the top level of `databricks.yml`.

Which fragment configures the target?

A. `targets: {dev: {host: https://dev.example.databricks.com, root_path: /Workspace/Shared/bundles/orders-dev}}`

B. `workspace: {dev: {host: https://dev.example.databricks.com, root_path: /Workspace/Shared/bundles/orders-dev}}`

C. `targets: {dev: {variables: {host: https://dev.example.databricks.com, root_path: /Workspace/Shared/bundles/orders-dev}}}}`

D. `targets: {dev: {workspace: {host: https://dev.example.databricks.com, root_path: /Workspace/Shared/bundles/orders-dev}}}}`

Correct answer: D

D. `targets: {dev: {workspace: {host: https://dev.example.databricks.com, root_path: /Workspace/Shared/bundles/orders-dev}}}}`

Explanation

Correct answer:

`targets: {dev: {workspace: {host: https://dev.example.databricks.com, root_path: /Workspace/Shared/bundles/orders-dev}}}}`

Why this is correct:

Bundle targets can override workspace settings, and the workspace-specific host and root_path belong beneath the target's workspace mapping. That selects the dev workspace URL and the requested deployment directory while leaving authentication to the configured profile.

Why the other options are wrong:

targets: `{dev: {host: https://dev.example.databricks.com, root_path: /Workspace/Shared/bundles/orders-dev}}`: Placing host and root_path directly on the target uses the wrong configuration nesting. These fields describe the target workspace and belong under workspace.

targets: `{dev: {variables: {host: https://dev.example.databricks.com, root_path: /Workspace/Shared/bundles/orders-dev}}}}`: variables supplies named values for interpolation; it does not

configure the target workspace connection or deployment root. Put these settings in the workspace block.

workspace: {dev: {host: https://dev.example.databricks.com, root_path: /Workspace/Shared/bundles/orders-dev}}: The top-level workspace mapping applies bundle-wide settings, not a named target override nested beneath dev. The question requires the host and root path specifically for the dev target.

References:

1. Configuration reference | Databricks on AWS:

<https://docs.databricks.com/aws/en/dev-tools/bundles/reference>

Question 33

In a bundle job, publish reads output from the refresh_pipeline task and must start only after that task succeeds. The fragment below is indented under publish.

```
task_key: publish
_____
notebook_task:
  notebook_path: ./src/publish.py
```

Which block completes the dependency?

- A. depends_on listing task_key refresh_pipeline, with run_if set to ALL_DONE, so publish starts once that task finishes and then reads its output
- B. run_if set to ALL_SUCCESS with no depends_on list, so publish waits until every other task in the job has succeeded before it starts
- C. depends_on listing task_key refresh_pipeline, with run_if left at its default of ALL_SUCCESS, so publish starts only after that task succeeds
- D. depends_on listing pipeline_id refresh_pipeline, with run_if left unset, so publish waits for that pipeline's latest update to succeed first

Correct answer: C

C. depends_on listing task_key refresh_pipeline, with run_if left at its default of ALL_SUCCESS, so publish starts only after that task succeeds

Explanation

Correct answer:

depends_on listing task_key refresh_pipeline, with run_if left at its default of ALL_SUCCESS, so publish starts only after that task succeeds

Why this is correct:

Lakeflow Jobs dependencies refer to another task by its task_key; a pipeline's own identifier is not the key used in the job DAG. With refresh_pipeline listed as the dependency, the default ALL_SUCCESS condition makes publish wait for that task's successful completion.

Why the other options are wrong:

depends_on listing pipeline_id refresh_pipeline, with run_if left unset, so publish waits for that pipeline's latest update to succeed first: depends_on takes task keys in the job, not a pipeline resource ID. The job waits for the pipeline task node to finish, regardless of the pipeline's underlying identifier.

depends_on listing task_key refresh_pipeline, with run_if set to ALL_DONE, so publish starts

once that task finishes and then reads its output: Because ALL_DONE ignores outcome, publish could begin after a failed refresh and consume stale results. The default success condition is necessary for this dependency.

run_if set to ALL_SUCCESS with no depends_on list, so publish waits until every other task in the job has succeeded before it starts: run_if defines a condition over dependencies; it does not create the dependency edges itself. With no depends_on, publish has no stated prerequisite to wait on.

References:

1. Configure task dependencies | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/jobs/run-if>

Question 34

A job runs extract, then load, then report, with report using Run if None failed. An operator canceled extract during today's run, and load then showed Upstream canceled.

What happens to report?

- A. It does not run, because a task marked Upstream canceled counts as failed when report's Run if condition is evaluated for this run
- B. It runs, because None failed looks only at direct dependencies, and load itself neither failed nor was canceled by the operator
- C. It runs, because a canceled upstream task counts as skipped, and the None failed condition ignores skipped dependencies entirely
- D. It waits for the operator to resume extract, because a canceled task pauses its downstream tasks until someone repairs the run

Correct answer: A

A. It does not run, because a task marked Upstream canceled counts as failed when report's Run if condition is evaluated for this run

Explanation

Correct answer:

It does not run, because a task marked Upstream canceled counts as failed when report's Run if condition is evaluated for this run

Why this is correct:

Lakeflow Jobs treats an upstream task marked Upstream canceled as failed when it evaluates downstream Run if conditions. The None failed condition therefore fails for report, so report is marked upstream-canceled/failed and is not started in this run.

Why the other options are wrong:

It runs, because None failed looks only at direct dependencies, and load itself neither failed nor was canceled by the operator: The condition is evaluated over the dependency graph's upstream task outcomes; the canceled extract remains part of the chain even though load is report's direct dependency. Its canceled state propagates as an unsuccessful dependency result.

It runs, because a canceled upstream task counts as skipped, and the None failed condition ignores skipped dependencies entirely: Canceled and excluded are distinct task states. An upstream-canceled task is evaluated as failed for Run if logic, not ignored like a skipped/excluded task.

It waits for the operator to resume extract, because a canceled task pauses its downstream tasks until someone repairs the run: Downstream tasks are evaluated from the current run state; they do not

wait for a canceled task to resume. Recovery requires a separate repair or new run.

References:

1. Configure task dependencies | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/jobs/run-if>

Question 35

An external database exposes only an ODBC endpoint, and only its ODBC driver is installed on notebook compute. Lakehouse Federation does not support it, no managed connector exists, and an approved Python ODBC client can query it.

How should the daily job land the extract in a Unity Catalog Delta table?

- A. Create a Lakehouse Federation connection to the endpoint, then run a daily CREATE OR REPLACE TABLE AS SELECT from the foreign catalog that it exposes
- B. Fetch the extract with the approved ODBC client inside the notebook, convert the result to a Spark DataFrame, and write it to the Delta table
- C. Read the endpoint with `spark.read.format("jdbc")` and the ODBC connection string, which Spark hands to the installed ODBC driver at runtime
- D. Point a daily COPY INTO command at the endpoint's connection string, which reads the extract and skips any rows that it loaded on earlier runs

Correct answer: B

B. Fetch the extract with the approved ODBC client inside the notebook, convert the result to a Spark DataFrame, and write it to the Delta table

Explanation

Correct answer:

Fetch the extract with the approved ODBC client inside the notebook, convert the result to a Spark DataFrame, and write it to the Delta table

Why this is correct:

The constraints eliminate supported federation, a managed connector, and a JDBC-accessible endpoint. The approved ODBC client is therefore the available source-side reader: fetch its result in Python, construct a Spark DataFrame from those records, and write that DataFrame into the governed Delta destination.

Why the other options are wrong:

Read the endpoint with `spark.read.format("jdbc")` and the ODBC connection string, which Spark hands to the installed ODBC driver at runtime: Spark's built-in `format("jdbc")` uses a JDBC driver and JDBC URL; an installed ODBC driver is a different interface and is not invoked by Spark JDBC this way. The scenario explicitly provides an ODBC client instead.

Create a Lakehouse Federation connection to the endpoint, then run a daily CREATE OR REPLACE TABLE AS SELECT from the foreign catalog that it exposes: Federation is explicitly unavailable for this endpoint, so it cannot create the foreign catalog needed by the proposed query. A SQL statement cannot overcome the missing connector support.

Point a daily COPY INTO command at the endpoint's connection string, which reads the extract and skips any rows that it loaded on earlier runs: COPY INTO reads supported files at a storage path; a database connection string is not a file source. Its file idempotency does not turn it into a database extraction connector.

References:

1. Spark data sources | Databricks on AWS:

<https://docs.databricks.com/aws/en/connect/spark-data-sources>

Question 36

CSV files contain `account_code` values such as 000417 that must keep their leading zeros, plus a balance column that must stay numeric. The current reader uses `inferSchema`, which turns `account_code` into an integer.

```
accounts = (spark.read.option("header", True)
            ._____
            .csv(source_path))
```

Which call completes the reader?

- A. `option("samplingRatio", 1.0)`, which infers types from every row instead of a sample, so codes with leading zeros are typed as strings
- B. `option("inferSchema", False)`, which reads `account_code` as text to keep the zeros while the balance column keeps its numeric type from inference
- C. `schema("account_code STRING, balance DOUBLE")`, which sets each column's type up front, so codes keep their zeros and balances stay numeric
- D. `option("primitivesAsString", True)`, which reads only values like 000417 as strings, while balances that look like decimals keep a numeric type

Correct answer: C

C. `schema("account_code STRING, balance DOUBLE")`, which sets each column's type up front, so codes keep their zeros and balances stay numeric

Explanation

Correct answer:

`schema("account_code STRING, balance DOUBLE")`, which sets each column's type up front, so codes keep their zeros and balances stay numeric

Why this is correct:

Supplying a schema sets the types before Spark parses the CSV values. Declaring `account_code` as `STRING` preserves characters such as leading zeros, while explicitly assigning `DOUBLE` to `balance` keeps that measurement numeric; schema inference is no longer making either decision.

Why the other options are wrong:

`option("inferSchema", False)`, which reads `account_code` as text to keep the zeros while the balance column keeps its numeric type from inference: With inference disabled, CSV fields default to strings rather than retaining numeric inference for balance. The code would preserve the account code but also turn the balance into text.

option("samplingRatio", 1.0), which infers types from every row instead of a sample, so codes with leading zeros are typed as strings: A full sampling ratio still asks Spark to infer types from observed values. A column containing digit-only values such as 000417 can still be inferred numerically and lose its formatting.

option("primitivesAsString", True), which reads only values like 000417 as strings, while balances that look like decimals keep a numeric type: primitivesAsString applies to JSON primitive parsing, not selective CSV column typing. It cannot convert only the account identifier while leaving balance numeric.

References:

1. `pyspark.sql.DataFrameReader.load` | PySpark documentation: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrameReader.load.html>

Question 37

In the Spark UI, a query's physical plan shows filter, select, groupBy, and join steps, and the join runs as a sort-merge join. The engineer wants to know which steps start a new stage because they need an Exchange.

Which steps create those boundaries?

- A. The groupBy and join steps, because rows that share a key must move to the same partition, which needs an Exchange between executors
- B. The groupBy, join, and select steps, because select reorders the columns, which also requires rows to move between the executors
- C. Only the groupBy step, because a sort-merge join reads both of its inputs in place and so never needs an Exchange between executors
- D. Only the join step, because groupBy combines rows inside each partition and so never needs an Exchange between executors afterward

Correct answer: A

A. The groupBy and join steps, because rows that share a key must move to the same partition, which needs an Exchange between executors

Explanation

Correct answer:

The groupBy and join steps, because rows that share a key must move to the same partition, which needs an Exchange between executors

Why this is correct:

Grouping by a key generally requires an Exchange so equal keys reach the same partition for aggregation. A sort-merge join also requires its inputs partitioned and sorted by join key, which normally adds Exchange boundaries; a simple filter, projection, or column selection does not itself shuffle rows.

Why the other options are wrong:

Only the groupBy step, because a sort-merge join reads both of its inputs in place and so never needs an Exchange between executors: The join must co-locate matching keys and sort its inputs, which normally requires Exchange stages on both sides. A sort-merge join does not read arbitrary partitions in place.

Only the join step, because groupBy combines rows inside each partition and so never needs an Exchange between executors afterward: The grouping step must bring matching keys together across input partitions before aggregation, so it commonly has an Exchange too. Only counting within pre-grouped partitions would avoid that shuffle, and no such premise is given.

The groupBy, join, and select steps, because select reorders the columns, which also requires rows to move between the executors: Selecting or reordering columns changes the row schema, not which executor owns each row. A projection alone does not require an Exchange.

References:

1. Adaptive query execution | Databricks on AWS:

<https://docs.databricks.com/aws/en/optimizations/age>

Question 38

A query in the Databricks SQL editor defines a named parameter `region_code`. Its value must filter a **STRING** column with a named parameter marker, without the query text being built through string substitution.

```
SELECT order_id, amount
FROM corp.sales.orders
WHERE region = _____;
```

Which expression completes the predicate?

- A. `:region_code`
- B. `?`
- C. `'${region_code}'`
- D. `{{ region_code }}`

Correct answer: A

A. `:region_code`

Explanation

Correct answer:

`:region_code`

Why this is correct:

Databricks SQL named value parameters use a colon followed by the parameter's name. `:region_code` is bound as a typed value when the query executes, leaving the SQL structure unchanged and avoiding unsafe text substitution.

Why the other options are wrong:

`{{ region_code }}`: Double braces are legacy mustache-style parameter syntax, rather than the named parameter marker requested in this query. Use the colon form to bind a value marker.

`?`: A question mark is an unnamed parameter marker; it does not name or associate itself with the `region_code` query parameter. This editor query specifically defines a named parameter.

`'${region_code}'`: `${...}` is not Databricks SQL's named parameter syntax. Quoting a substituted string

would construct query text rather than bind the value as a marker.

References:

1. Use named parameter markers | Databricks on AWS:

<https://docs.databricks.com/aws/en/sql/user/queries/query-parameters>

Question 39

A bundle project has a dev target. Job 8421 already exists in that workspace, with schedules and run history the team must keep. The engineer wants to generate its bundle definition and link it to the existing job in one step.

Which command does this?

- A. databricks bundle deployment bind 8421 on its own, which writes the job's definition into resources/ and links it to the job
- B. databricks bundle generate job --existing-job-id 8421 --bind, which writes the job's definition locally and links it to that job
- C. databricks bundle deploy -t dev --job-id 8421, which writes the job's YAML definition locally and deploys it over the existing job
- D. databricks jobs get 8421 with its JSON output saved under resources/, which the next bundle deploy then links to the existing job

Correct answer: B

B. databricks bundle generate job --existing-job-id 8421 --bind, which writes the job's definition locally and links it to that job

Explanation

Correct answer:

databricks bundle generate job --existing-job-id 8421 --bind, which writes the job's definition locally and links it to that job

Why this is correct:

bundle generate job --existing-job-id reads the existing job and writes its resource configuration into the bundle project. Adding --bind links that generated resource to the existing workspace job, allowing later deployments to manage it without recreating it and losing its identity or run history.

Why the other options are wrong:

databricks jobs get 8421 with its JSON output saved under resources/, which the next bundle deploy then links to the existing job: jobs get retrieves API JSON but does not generate a bundle resource file or bind the job into bundle state. Manual conversion and a later deploy are separate work.

databricks bundle deployment bind 8421 on its own, which writes the job's definition into resources/ and links it to the job: bundle deployment bind is not the one-step generate-and-bind command described. Binding an existing resource presupposes a corresponding bundle resource definition.

databricks bundle deploy -t dev --job-id 8421, which writes the job's YAML definition locally and

deploys it over the existing job: bundle deploy applies an existing local resource definition; it does not generate one from a job ID. The --job-id option does not perform the requested generate-and-bind flow.

References:

1. bundle command group | Databricks on AWS:

<https://docs.databricks.com/aws/en/dev-tools/cli/bundle-commands>

Question 40

A task must make at most three attempts in total, and an attempt that exceeds the task's timeout must be retried the same way as a failed attempt.

```
task_key: extract
timeout_seconds: 3600

_____
notebook_task:
  notebook_path: ./extract.py
```

Which settings complete the task configuration?

- A.** `max_retries: 2` with `retry_on_timeout` left at its default, so failed and timed-out attempts are each retried at most twice
- B.** `max_retries: 3` with `retry_on_timeout` left unset, which counts the first execution as one retry, for three attempts in all
- C.** `max_retries: 2` with `retry_on_timeout: true`, so a timed-out or failed attempt is retried at most twice, three attempts in all
- D.** `max_retries: 3` with `retry_on_timeout: true`, so a timed-out or failed attempt is retried until three attempts in total have run

Correct answer: C

C. `max_retries: 2` with `retry_on_timeout: true`, so a timed-out or failed attempt is retried at most twice, three attempts in all

Explanation

Correct answer:

`max_retries: 2` with `retry_on_timeout: true`, so a timed-out or failed attempt is retried at most twice, three attempts in all

Why this is correct:

`max_retries` counts retries after the original attempt, so a value of two permits at most three total attempts. Setting `retry_on_timeout: true` includes a timed-out attempt in that retry policy, matching both the attempt ceiling and timeout behavior.

Why the other options are wrong:

`max_retries: 3` with `retry_on_timeout` left unset, which counts the first execution as one retry, for three attempts in all: Three retries means one initial attempt plus as many as three more, allowing four total attempts. The question caps total attempts at three.

`max_retries: 2` with `retry_on_timeout` left at its default, so failed and timed-out attempts are each retried

at most twice: Two retries gives the right attempt count, but leaving `retry_on_timeout` unset does not explicitly require timeouts to be retried. The stem specifically includes timeout failures.

max_retries: 3 with `retry_on_timeout: true`, so a timed-out or failed attempt is retried until three attempts in total have run: With three retries, the initial execution can be followed by three additional attempts, exceeding the three-attempt total. The retry count must be two.

References:

1. Configure and edit tasks in Lakeflow Jobs | Databricks on AWS:

<https://docs.databricks.com/aws/en/jobs/configure-task>

Question 41

A large events DataFrame joins a small lookup DataFrame with a broadcast hash join. The lookup fits comfortably in executor memory, and the join key is not skewed.

How does broadcasting the lookup reduce data movement?

- A. The events rows are hash-partitioned on the join key while the lookup stays on the driver, so only one side moves across the network
- B. The lookup is hash-partitioned across executors by key, so each events partition probes its matches remotely without moving its rows
- C. Each executor receives a full copy of the lookup, so the events rows join against it where they already sit, with no shuffle of events
- D. The lookup is collected to the driver and the events rows are sent there to be probed, so no exchange between executors is needed

Correct answer: C

C. Each executor receives a full copy of the lookup, so the events rows join against it where they already sit, with no shuffle of events

Explanation

Correct answer:

Each executor receives a full copy of the lookup, so the events rows join against it where they already sit, with no shuffle of events

Why this is correct:

A broadcast hash join distributes a full copy of the small build-side lookup to the executors. Each executor can then join its existing partition of the large events table locally, avoiding a network shuffle of those fact rows by join key.

Why the other options are wrong:

The events rows are hash-partitioned on the join key while the lookup stays on the driver, so only one side moves across the network: The broadcast copy is sent to the executors, not kept solely on the driver. If the lookup remained on the driver, workers could not perform the local hash probes described by the broadcast join.

The lookup is hash-partitioned across executors by key, so each events partition probes its matches remotely without moving its rows: Sharding lookup records by key involves repartitioning the relation and moving rows over the network. Replication avoids that exchange because the lookup fits in executor memory.

The lookup is collected to the driver and the events rows are sent there to be probed, so no

exchange between executors is needed: The driver coordinates the query but does not collect and probe all event rows for a broadcast hash join. Sending the large relation to the driver would defeat distributed execution and risk driver memory exhaustion.

References:

1. Performance Tuning | Apache Spark: <https://spark.apache.org/docs/latest/sql-performance-tuning>

Question 42

An engineer pulls main into her Git folder, and the pull stops with a merge conflict in one notebook. She wants to keep some of her lines and some of the incoming lines.

What should she do?

- A. Choose Keep all current in the conflict view, which resolves the file by keeping her version of every conflicting line in it
- B. Abort the merge and pull again with rebase, which replays her commits on main and combines the conflicting lines without asking her
- C. Choose Take all incoming in the conflict view, which resolves the file by keeping the incoming version of every conflicting line
- D. Edit the conflicted file in the Git folder, choose lines from both sides, delete the conflict markers, and then mark it as resolved

Correct answer: D

D. Edit the conflicted file in the Git folder, choose lines from both sides, delete the conflict markers, and then mark it as resolved

Explanation

Correct answer:

Edit the conflicted file in the Git folder, choose lines from both sides, delete the conflict markers, and then mark it as resolved

Why this is correct:

A merge conflict marks regions where the current branch and incoming branch both changed overlapping content. To keep selected lines from each version, edit the conflicted notebook/file, remove the conflict markers, and mark the resolved file before completing the merge.

Why the other options are wrong:

Choose Take all incoming in the conflict view, which resolves the file by keeping the incoming version of every conflicting line: Taking all incoming selects one side for the conflicted region and discards the current-side edits there. It does not combine chosen lines from both versions.

Abort the merge and pull again with rebase, which replays her commits on main and combines the conflicting lines without asking her: Rebasing changes how commits are replayed, but it does not automatically decide how to combine conflicting lines. The conflict still needs a deliberate resolution.

Choose Keep all current in the conflict view, which resolves the file by keeping her version of every conflicting line in it: Keeping all current preserves the local side and discards incoming edits in the conflict. The request is to retain selected content from both sides.

References:

1. Create and manage Git folders | Databricks on AWS:

<https://docs.databricks.com/aws/en/repos/git-operations-with-repos>

Question 43

An engineer wants to find which of a job's twelve tasks drove a slow increase in run time over the last four months. The job's run history in the UI no longer covers the whole period.

Which source should be queried?

- A. The job cluster's Spark UI, whose stage timeline keeps the task durations for each run of the job from the past four months
- B. DESCRIBE HISTORY on each output table, whose operation metrics show which task's writes took longer in each of those months
- C. system.lakeflow.job_run_timeline on its own, whose rows record each task's start and end time for each run of the job
- D. system.lakeflow.job_task_run_timeline, whose task-level run periods can be grouped by task key and compared across those months

Correct answer: D

D. system.lakeflow.job_task_run_timeline, whose task-level run periods can be grouped by task key and compared across those months

Explanation

Correct answer:

system.lakeflow.job_task_run_timeline, whose task-level run periods can be grouped by task key and compared across those months

Why this is correct:

system.lakeflow.job_task_run_timeline contains task-level run periods and related metadata, with 365 days of retention. That provides individual task durations across the requested four-month span; grouping by task key lets the engineer compare each task's trend.

Why the other options are wrong:

DESCRIBE HISTORY on each output table, whose operation metrics show which task's writes took longer in each of those months: Delta history describes operations on output tables and their transaction metadata, not elapsed time for every job task. It cannot consistently attribute run duration changes to a task.

system.lakeflow.job_run_timeline on its own, whose rows record each task's start and end time for each run of the job: job_run_timeline records job-level periods, not the task-level durations needed to isolate one of twelve tasks. Use the task timeline table for that granularity.

The job cluster's Spark UI, whose stage timeline keeps the task durations for each run of the job from the past four months: The Spark UI focuses on a specific Spark application and does not

guarantee four months of retained job-task history. The system task timeline is the durable queryable source here.

References:

1. Jobs system table reference | Databricks on AWS:

<https://docs.databricks.com/aws/en/admin/system-tables/jobs>

Question 44

user1 belongs to the analysts group, which holds SELECT on corp.sales.orders. The table owner then runs the statements below, and user1 stays a member of analysts.

```
GRANT SELECT ON TABLE corp.sales.orders TO user1;  
REVOKE SELECT ON TABLE corp.sales.orders FROM user1;
```

Can user1 still query the table?

- A. No, because revoking SELECT from user1 removes that privilege from every source, including the grant that the analysts group holds
- B. Yes, because a REVOKE on a table takes effect only after the table owner also revokes SELECT from the table's parent schema
- C. No, as a revoke on a user overrides the grants to groups that the user belongs to until the owner reissues the group's grant
- D. Yes, because the revoke removes only the direct grant to user1, while SELECT still reaches user1 through the analysts group's grant

Correct answer: D

D. Yes, because the revoke removes only the direct grant to user1, while SELECT still reaches user1 through the analysts group's grant

Explanation

Correct answer:

Yes, because the revoke removes only the direct grant to user1, while SELECT still reaches user1 through the analysts group's grant

Why this is correct:

Unity Catalog grants are additive across the user's direct privileges and privileges inherited from groups. Revoking the direct SELECT grant removes that one assignment; because user1 remains in analysts, its SELECT grant on the table still authorizes the query (assuming required parent usage privileges).

Why the other options are wrong:

No, because revoking SELECT from user1 removes that privilege from every source, including the grant that the analysts group holds: A direct revoke changes the named principal's grant and does not erase the separate assignment to the group. Group membership continues to contribute its own privileges.

Yes, because a REVOKE on a table takes effect only after the table owner also revokes SELECT from the table's parent schema: The revoke takes effect on the direct grant without an additional

schema-level revocation. It also does not affect the group's independent SELECT assignment.

No, as a revoke on a user overrides the grants to groups that the user belongs to until the owner reissues the group's grant: Unity Catalog does not turn a user-level revoke into a deny that overrides grants received through group membership. Removing inherited access would require changing the group grant or membership.

References:

1. Manage privileges in Unity Catalog | Databricks on AWS:

<https://docs.databricks.com/aws/en/data-governance/unity-catalog/manage-privileges/>

Question 45

A streaming job reads the Delta table `silver.orders` and appends to `gold`. A nightly GDPR process now runs `DELETE` statements on `silver.orders`, and the stream fails after each run. Deleted rows do not need to propagate downstream.

Which change keeps the stream running?

- A. Switch the stream to `trigger(availableNow=True)`, so that it processes the `DELETE` commits as a batch instead of failing on them
- B. Set the `skipChangeCommits` option on the stream, because it then ignores commits that update or delete rows and keeps reading appends
- C. Restart the stream with a new checkpoint after each `DELETE`, letting it read `silver.orders` again starting from the table's current version
- D. Enable change data feed on `silver.orders`, so that the stream reads the deletes as normal appends without any other change to the query

Correct answer: B

B. Set the `skipChangeCommits` option on the stream, because it then ignores commits that update or delete rows and keeps reading appends

Explanation

Correct answer:

Set the `skipChangeCommits` option on the stream, because it then ignores commits that update or delete rows and keeps reading appends

Why this is correct:

Delta streaming reads normally expect append-only source commits, so an upstream `DELETE` causes the query to fail. `skipChangeCommits` ignores transactions that modify or delete existing rows and continues processing append commits, which fits because downstream deletion propagation is explicitly unnecessary.

Why the other options are wrong:

Restart the stream with a new checkpoint after each `DELETE`, letting it read `silver.orders` again starting from the table's current version: Discarding checkpoint progress can cause replay or gaps, and it does not make future change commits acceptable. Configure the source to skip those commits while preserving its offsets.

Switch the stream to `trigger(availableNow=True)`, so that it processes the `DELETE` commits as a batch instead of failing on them: `availableNow` changes when the stream stops after processing available input; it does not make a Delta source accept `DELETE` commits. The source change still needs explicit handling.

Enable change data feed on silver.orders, so that the stream reads the deletes as normal appends without any other change to the query: Change data feed exposes row-level changes, but the query would have to read and process that feed using CDC semantics. Enabling it alone does not make deletes arrive as ordinary appends.

References:

1. Delta Lake table streaming reads and writes | Databricks on AWS:

<https://docs.databricks.com/aws/en/structured-streaming/delta-lake>

Question 46

An ABAC column mask policy on catalog corp should apply only to tables tagged sensitivity = high, masking their columns tagged pii. Tables without that table-level tag must stay unmasked.

Which clause scopes the policy to those tables?

- A. EXCEPT has_tag_value('sensitivity', 'low'), which exempts each table whose table-level sensitivity tag is set to low
- B. MATCH COLUMNS has_tag_value('sensitivity', 'high'), which limits the policy to the tables that carry that table-level tag value
- C. USING COLUMNS (sensitivity), which passes each table's tag value to the mask function so that it can decide per table
- D. WHEN has_tag_value('sensitivity', 'high'), which limits the policy to the tables that carry that table-level tag value

Correct answer: D

D. WHEN has_tag_value('sensitivity', 'high'), which limits the policy to the tables that carry that table-level tag value

Explanation

Correct answer:

WHEN has_tag_value('sensitivity', 'high'), which limits the policy to the tables that carry that table-level tag value

Why this is correct:

In an ABAC column-mask policy, WHEN defines the condition on the table or securable object for the policy to apply. Testing sensitivity=high there scopes the policy to qualifying tables; MATCH COLUMNS can separately select their pii columns for masking.

Why the other options are wrong:

MATCH COLUMNS has_tag_value('sensitivity', 'high'), which limits the policy to the tables that carry that table-level tag value: The policy needs a condition on the tagged table, while MATCH COLUMNS identifies tagged columns. The sensitivity test belongs to the table-level WHEN clause.

USING COLUMNS (sensitivity), which passes each table's tag value to the mask function so that it can decide per table: Additional column arguments to a mask function are handled by USING COLUMNS; that clause does not scope a policy by table tags. Use the policy condition for the sensitivity attribute.

EXCEPT has_tag_value('sensitivity', 'low'), which exempts each table whose table-level sensitivity tag is set to low: EXCEPT is not the clause that tests a table tag for ABAC application. Excluding low

sensitivity would not state the required positive condition that the value be high.

References:

1. Create and manage row filter and column mask policies | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/data-governance/unity-catalog/abac/policies>

Question 47

After a pipeline refreshes the gold tables each morning, a published AI/BI dashboard must show the new numbers and email its subscribers, without anyone opening it by hand.

Which task should follow the pipeline task in the job?

- A. A SQL task that runs each of the dashboard's dataset queries on a SQL warehouse, which refreshes the published dashboard's cached results
- B. A notebook task that calls the dashboard's URL through an HTTP request, which refreshes the published dashboard and emails its subscribers
- C. A dashboard task that refreshes the published dashboard on a SQL warehouse and can send the refreshed version to the dashboard's subscribers
- D. A Run Job task that starts the dashboard's own refresh schedule, which updates the published dashboard and then notifies its subscribers

Correct answer: C

C. A dashboard task that refreshes the published dashboard on a SQL warehouse and can send the refreshed version to the dashboard's subscribers

Explanation

Correct answer:

A dashboard task that refreshes the published dashboard on a SQL warehouse and can send the refreshed version to the dashboard's subscribers

Why this is correct:

A Dashboard task refreshes a published dashboard as a job-graph task and can select subscribers to receive email notifications when that refresh completes. Placing it after the pipeline task makes the refresh follow the new Gold data without requiring a person to open the dashboard.

Why the other options are wrong:

A SQL task that runs each of the dashboard's dataset queries on a SQL warehouse, which refreshes the published dashboard's cached results: Running dataset SQL queries does not issue the dashboard refresh or subscriber notification. It recomputes query results but leaves the published dashboard's job task behavior unconfigured.

A Run Job task that starts the dashboard's own refresh schedule, which updates the published dashboard and then notifies its subscribers: Starting a second job only orchestrates another job resource; it does not invoke the dashboard refresh operation or deliver subscriber emails.

A notebook task that calls the dashboard's URL through an HTTP request, which refreshes the published dashboard and emails its subscribers: A dashboard URL is for access and viewing; calling

it does not invoke the job task operation that refreshes results and sends subscriber notifications.

References:

1. Dashboard task for jobs | Databricks on AWS:

<https://docs.databricks.com/aws/en/jobs/tasks/dashboard>

Question 48

An operations feed arrives all day, and alerts must reflect new events within seconds. Finance receives a full daily snapshot at 01:00 and needs a report by 06:00. The team avoids continuous compute where it adds nothing.

Which design fits both workloads?

- A. A scheduled batch job every minute for the operations alerts, and a continuous stream that processes the daily finance snapshot
- B. A continuously running stream for the operations alerts, and a scheduled batch job after 01:00 that builds the finance report
- C. A stream with an availableNow trigger scheduled every minute for the alerts, and a scheduled batch job after 01:00 for the report
- D. A continuous stream for the operations alerts, and a second continuous stream that reads the finance snapshot as soon as it lands

Correct answer: B

B. A continuously running stream for the operations alerts, and a scheduled batch job after 01:00 that builds the finance report

Explanation

Correct answer:

A continuously running stream for the operations alerts, and a scheduled batch job after 01:00 that builds the finance report

Why this is correct:

The alerts workload has a seconds-level freshness target throughout the day, so a continuously running stream can process new events as they arrive. The finance input is a once-daily complete snapshot with a five-hour reporting window, making one scheduled batch after 01:00 sufficient without idle continuous compute.

Why the other options are wrong:

A continuous stream for the operations alerts, and a second continuous stream that reads the finance snapshot as soon as it lands: Running finance continuously offers no useful cadence for a complete daily snapshot and consumes compute between arrivals. Schedule batch work after the known delivery time.

A scheduled batch job every minute for the operations alerts, and a continuous stream that processes the daily finance snapshot: A one-minute batch cadence cannot guarantee the seconds-level alert freshness, while keeping a stream continuously active for one daily snapshot is unnecessary.

A stream with an availableNow trigger scheduled every minute for the alerts, and a scheduled batch job after 01:00 for the report: availableNow drains available input and stops; scheduling it every minute still introduces polling gaps and cannot meet a seconds-level latency goal. Reserve it for bounded periodic processing.

References:

1. Structured Streaming Programming Guide | Apache Spark:

<https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>

Question 49

Files for one delivery arrive over several minutes. A file arrival trigger starts a run as soon as the first file lands, which creates many small runs. The team wants one run after the files stop arriving.

Which trigger setting addresses this?

- A. Maximum concurrent runs set to 1, which merges the many small runs into one run that then processes all of the delivery's files together
- B. Minimum time between triggers, which starts a run as soon as the first file lands and then waits the configured period before the next one
- C. Wait after last change, because it starts a run only after no new files have arrived at the monitored location for the configured period
- D. A job timeout of several minutes, which holds the first run open until the remaining files of the delivery have landed at the location

Correct answer: C

C. Wait after last change, because it starts a run only after no new files have arrived at the monitored location for the configured period

Explanation

Correct answer:

Wait after last change, because it starts a run only after no new files have arrived at the monitored location for the configured period

Why this is correct:

Wait after last change is a debounce interval: every new file resets its timer, and the job starts only after the configured quiet period. Choosing an interval longer than the usual gap between files batches the delivery into one run after arrivals stop.

Why the other options are wrong:

Maximum concurrent runs set to 1, which merges the many small runs into one run that then processes all of the delivery's files together: Maximum concurrency prevents overlapping runs; it does not merge separate trigger events into a single queued run. Additional arrivals may still result in later runs.

Minimum time between triggers, which starts a run as soon as the first file lands and then waits the configured period before the next one: A cooldown limits how frequently runs can start, but it does not reset its timer on every arriving file. It can still launch partway through the current delivery.

A job timeout of several minutes, which holds the first run open until the remaining files of the

delivery have landed at the location: A job timeout terminates a run that has exceeded its allowed execution duration. It does not keep a run open waiting for files that have not yet arrived.

References:

1. Trigger jobs when new files arrive | Databricks on AWS:

<https://docs.databricks.com/aws/en/jobs/file-arrival-triggers>

Question 50

A Lakeflow Spark Declarative Pipelines SQL pipeline reads `silver_events`, an append-only table with category and a unique `event_id`. Gold must hold one running count per category, with no event-time windows, and categories are few.

Which definition fits gold?

- A. A streaming table that appends one row per category on each update, so that readers sum the rows to get the running counts
- B. A materialized view that groups `silver_events` by category and counts rows, which the pipeline keeps current on every update
- C. A temporary view that groups `silver_events` by category, which stores its counts between pipeline updates for later readers
- D. A streaming table that reads `silver_events` and overwrites itself with fresh counts on each micro-batch that it processes

Correct answer: B

B. A materialized view that groups `silver_events` by category and counts rows, which the pipeline keeps current on every update

Explanation

Correct answer:

A materialized view that groups `silver_events` by category and counts rows, which the pipeline keeps current on every update

Why this is correct:

A materialized view stores the result of its defining batch query and can maintain that result as the append-only source changes. Grouping by the small category domain yields one persisted count row per category without inventing event-time windows or accumulating duplicate count rows.

Why the other options are wrong:

A temporary view that groups `silver_events` by category, which stores its counts between pipeline updates for later readers: A temporary view stores a query definition scoped to its session, not persisted counts available across pipeline updates. It cannot serve as the durable Gold object described.

A streaming table that reads `silver_events` and overwrites itself with fresh counts on each micro-batch that it processes: A streaming table is designed to process incoming records as a stream, not to replace itself with an aggregate snapshot on every micro-batch. The question asks for a maintained grouped result.

A streaming table that appends one row per category on each update, so that readers sum the

rows to get the running counts: Accumulating each update's aggregate as an append produces multiple totals per category, not one current count. A materialized view exposes the maintained aggregate result.

References:

1. Use standalone materialized views | Databricks on AWS:

<https://docs.databricks.com/aws/en/ldp/dbsql/materialized>

Question 51

A governed table has a row filter and a column mask applied. A user who is subject to both runs `SELECT *` on the table.

Which TWO statements describe the result?

- A. The column mask rewrites the values stored in the table's data files, so earlier versions read through time travel also return masked values
- B. The column mask is skipped for `SELECT *`, because a mask applies when the query names the masked column explicitly
- C. The row filter raises a permission error when the table holds rows that its function rejects for the user running the query
- D. The row filter leaves out the rows for which its function returns `FALSE`, and the query then completes without raising any error
- E. The column mask can replace sensitive values in the result while the masked column itself stays in the query's output schema

Correct answers: D, E

- D. The row filter leaves out the rows for which its function returns `FALSE`, and the query then completes without raising any error
- E. The column mask can replace sensitive values in the result while the masked column itself stays in the query's output schema

Explanation

Correct answer:

The row filter leaves out the rows for which its function returns `FALSE`, and the query then completes without raising any error

The column mask can replace sensitive values in the result while the masked column itself stays in the query's output schema

Why this is correct:

Row filters and column masks are query-time access controls: the row filter excludes rows whose predicate is false, while the mask substitutes returned values for the masked column. The mask changes the result value, not the column's presence in the result schema or the underlying stored data.

Why the other options are wrong:

The column mask rewrites the values stored in the table's data files, so earlier versions read through time travel also return masked values: Masking changes the value presented to a reader at query time; it does not rewrite stored Parquet records or historical Delta snapshots.

The row filter raises a permission error when the table holds rows that its function rejects for the user running the query: Rejected rows are filtered out of the result rather than causing a permission error. The policy is designed to limit visibility while allowing the authorized query to complete.

The column mask is skipped for SELECT *, because a mask applies when the query names the masked column explicitly: Masks apply to the governed column whenever it is returned, including through SELECT *. Explicitly naming the column is not a condition for enforcement.

References:

1. Row filters and column masks | Databricks on AWS:

<https://docs.databricks.com/aws/en/data-governance/unity-catalog/filters-and-masks>

Question 52

A task named `count_new` sets a task value named `row_count`. The publish task must run only when `row_count` is greater than zero; otherwise the job should skip publishing and still finish successfully.

How should the job decide whether publish runs?

- A. Add a For each task over `{{tasks.count_new.values.row_count}}`, whose nested publish task runs once per row and never when it is 0
- B. Add an If/else task comparing `{{tasks.count_new.values.row_count}}` with 0 using `>`, and make publish depend on its true outcome
- C. Make publish depend on `count_new` with `run_if` set to `{{tasks.count_new.values.row_count}} > 0`, so publish runs only for a positive count
- D. Have `count_new` raise an error when `row_count` is 0, and make publish depend on it with `run_if: ALL_SUCCESS` so that publishing is skipped

Correct answer: B

B. Add an If/else task comparing `{{tasks.count_new.values.row_count}}` with 0 using `>`, and make publish depend on its true outcome

Explanation

Correct answer:

Add an If/else task comparing `{{tasks.count_new.values.row_count}}` with 0 using `>`, and make publish depend on its true outcome

Why this is correct:

The If/else task evaluates a Boolean expression using the upstream task value, so it can branch when `row_count > 0`. Making publish depend on the true branch means a zero count skips only that branch; the condition itself does not need to fail the job.

Why the other options are wrong:

Add a For each task over `{{tasks.count_new.values.row_count}}`, whose nested publish task runs once per row and never when it is 0: A For each task expects an array of items to iterate over, not a scalar count for a yes/no decision. Looping over a count would also represent repeated work rather than a single publish gate.

Have `count_new` raise an error when `row_count` is 0, and make publish depend on it with `run_if: ALL_SUCCESS` so that publishing is skipped: Raising an error marks the producing task failed and risks failing the workflow; it does not express a successful no-data outcome. A conditional branch is designed to skip publishing without manufacturing a failure.

Make publish depend on count_new with run_if set to `{{tasks.count_new.values.row_count}} > 0`, so publish runs only for a positive count: run_if selects among task completion states such as success or failure; it does not evaluate arbitrary value comparisons. Compare the task value in an If/else condition task instead.

References:

1. Add branching logic to a job with the If/else task | Databricks on AWS:

<https://docs.databricks.com/gcp/en/jobs/tasks/if-else>

Question 53

A partner reporting tool outside Databricks can read Delta tables only by storage path, using an IAM role with read access to the whole corp-fin bucket. It cannot call Unity Catalog APIs, and Compatibility Mode is not enabled. DESCRIBE TABLE EXTENDED shows:

```
corp.fin.ledger
  Type      MANAGED
  Provider   delta
  Location   s3://corp-fin/uc/__unitystorage/schemas/4be1.../tables/9c07...
corp.fin.fx_rates
  Type      EXTERNAL
  Provider   delta
  Location   s3://corp-fin/feeds/fx_rates
```

Which access plan is supported?

- A. Read ledger by path, since managed storage under __unitystorage is the location that Unity Catalog exposes to the readers outside of Databricks
- B. Read both tables by path, since both Locations sit in the corp-fin bucket and the tool's IAM role already grants read access to that whole bucket
- C. Read neither table by path, since Unity Catalog privileges are enforced on file reads of both paths, and the tool cannot call Unity Catalog APIs
- D. Read fx_rates by path but not ledger, since the files of a managed table are meant to be reached only through the governed Unity Catalog access

Correct answer: D

D. Read fx_rates by path but not ledger, since the files of a managed table are meant to be reached only through the governed Unity Catalog access

Explanation

Correct answer

Read fx_rates by path but not ledger, since the files of a managed table are meant to be reached only through the governed Unity Catalog access

Why this is correct

The table type decides which paths an outside tool may use. Path-based access to a Unity Catalog managed table is not supported (outside Compatibility Mode) because it bypasses Unity Catalog access controls; external engines reach managed tables through Unity Catalog's open APIs with credential vending, which this tool cannot call. fx_rates is external, and external tables are the documented choice when non-Databricks clients need direct access to the files, so the tool's IAM role can read that path.

Why the other options are wrong

Read both tables by path, since both Locations sit in the corp-fin bucket and the tool's IAM role already grants read access to that whole bucket - Bucket-level IAM access makes ledger's files physically reachable, but reading a managed table by path is unsupported because it sidesteps Unity Catalog governance. Living in the company's own bucket does not turn a managed table into an external one.

Read ledger by path, since managed storage under __unitystorage is the location that Unity Catalog exposes to the readers outside of Databricks - The __unitystorage directory is internal managed storage, not an endpoint for outside readers. Engines outside Databricks reach managed tables through the Unity REST API or the Iceberg REST catalog with vended credentials, neither of which the tool supports.

Read neither table by path, since Unity Catalog privileges are enforced on file reads of both paths, and the tool cannot call Unity Catalog APIs - Direct file reads of an external table's path go around Unity Catalog altogether; its privileges are not enforced when external systems access the data files. The tool's own IAM role is enough to read fx_rates.

References

- [Unity Catalog managed tables for Delta Lake and Apache Iceberg](#)
- [Work with external tables](#)

Question 54

A notebook builds a filtered DataFrame from a large Delta table, where the filter is the expensive part. Two counts, a write, and a display then run on it, and the filtered data fits in cluster memory.

Which change should be tested first, then remeasured?

- A. Call `repartition(200)` on the filtered DataFrame before its first action, so each later action reads balanced partitions and recomputes less
- B. Call `cache()` on the unfiltered source DataFrame before the first action, so each later action reuses the raw rows and reapplies only the filter
- C. Enable the disk cache on the cluster, so each action reads the Delta files from local SSD instead of cloud storage and recomputes faster
- D. Call `cache()` on the filtered DataFrame before its first action, so the later actions reuse the stored filtered rows instead of recomputing them

Correct answer: D

D. Call `cache()` on the filtered DataFrame before its first action, so the later actions reuse the stored filtered rows instead of recomputing them

Explanation

Correct answer:

Call `cache()` on the filtered DataFrame before its first action, so the later actions reuse the stored filtered rows instead of recomputing them

Why this is correct:

Spark transformations are lazy, so each action can otherwise recompute the filtered source plan. Caching the filtered DataFrame before its first action lets the later counts, write, and display reuse the materialized filtered rows; remeasure to see whether the saved computation outweighs cache cost.

Why the other options are wrong:

Enable the disk cache on the cluster, so each action reads the Delta files from local SSD instead of cloud storage and recomputes faster: Disk cache stores local copies of source files to speed later reads, but the expensive filter is still part of the query plan. The filtered DataFrame cache directly targets repeated evaluation of that transformation.

Call `cache()` on the unfiltered source DataFrame before the first action, so each later action reuses the raw rows and reapplies only the filter: Caching the unfiltered source stores more rows than needed and still leaves filtering to be applied for downstream actions. Cache after the expensive filter so reuse covers the costly result.

Call `repartition(200)` on the filtered DataFrame before its first action, so each later action reads

balanced partitions and recomputes less: repartition(200) redistributes rows but does not persist them. Each action can still rerun the filter and shuffle, so repartitioning alone does not eliminate repeated computation.

References:

1. cache | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/pyspark/reference/classes/dataframe/cache>

Question 55

A report combines three tables that separate pipelines update at unpredictable times. It must go out by 08:00 every day, even when one of the tables did not change overnight.

Which trigger fits the report job?

- A. A table update trigger on the three tables with All tables are updated, which fires once each of the three tables has a new commit
- B. A continuous trigger on the report job, which keeps rerunning the report so that the latest version is ready whenever 08:00 arrives
- C. A scheduled trigger at a fixed time before 08:00, which runs the report whether or not each of the three tables has changed
- D. A table update trigger on the three tables with Any table is updated, which fires on the first commit and again on each later one

Correct answer: C

C. A scheduled trigger at a fixed time before 08:00, which runs the report whether or not each of the three tables has changed

Explanation

Correct answer:

A scheduled trigger at a fixed time before 08:00, which runs the report whether or not each of the three tables has changed

Why this is correct:

A scheduled trigger runs at a chosen time regardless of whether upstream tables changed. That meets the fixed daily delivery deadline and ensures the report still runs when one pipeline had no overnight updates; the schedule can be set early enough to finish by 08:00.

Why the other options are wrong:

A continuous trigger on the report job, which keeps rerunning the report so that the latest version is ready whenever 08:00 arrives: A continuous trigger starts another run after the report job completes or fails. It keeps compute active and does not target one report run at a predictable time.

A table update trigger on the three tables with All tables are updated, which fires once each of the three tables has a new commit: Requiring a fresh commit from every source means one unchanged table can suppress the whole report run. A deadline-driven report should not depend on every input changing.

A table update trigger on the three tables with Any table is updated, which fires on the first commit and again on each later one: An any-table-updated trigger responds to changes, not the

deadline, and can launch multiple times as separate pipelines update. It may also fail to run when there are no changes.

References:

1. Automate jobs with schedules and triggers | Databricks on AWS:

<https://docs.databricks.com/aws/en/jobs/triggers>